Cluster Computing
https://doi.org/10.1007/s10586-024-04518-z

Reinforcement learning based task offloading of IoT applications in fog computing: algorithms and optimization techniques

Takwa Allaoui[1] · Kaouther Gasmi[1] · Tahar Ezzedine[1]

Received: 25 October 2023 / Revised: 10 April 2024 / Accepted: 16 April 2024

Abstract

In recent years, fog computing has become a promising technology that supports computationally intensive and time-sensitive applications, especially when dealing with Internet of Things (IoT) devices with limited processing capability. In this context, offloading can push resource-intensive tasks closer to the end devices at the network edge. This allows user equipment to profit from the fog computing environment by offloading their tasks to fog resources. Thus, computation offloading mechanisms can overcome the resource constraints of devices and enhance the system's performance by minimizing delay and extending the battery lifetime of devices. In this regard, designing an algorithm to decide which tasks to offload and where to execute them is crucial. Recently, there has been a growing interest in utilizing Reinforcement Learning (RL) and deep reinforcement learning (DRL), to address computation offloading mechanisms in the context of fog computing. This paper reviews the research conducted on Reinforcement learning (RL) and Deep Reinforcement Learning (DRL) based computation offloading mechanisms for IoT applications in the fog environment. We provide a comprehensive and detailed survey, analyzing and classifying the research paper in terms of RL techniques, objectives, architecture, and use cases. Then, in particular, we identify the advantages and weaknesses of each paper. After that, We systematically elaborate on open issues and future research directions that are crucial for the next decade.

Keywords Fog computing · Computation offloading · Reinforcement learning · Deep reinforcement learning · Metrics

1 Introduction

It has been estimated that the number of deployed smart devices will reach 75 billion by 2025, while the amount of data generated by these devices is expected to increase to striking 2.3 zettabytes per year, as opposed to 1.1 zettabytes of data per year estimated in 2016 [1]. One of the biggest challenges in the ecosystem of the Internet of Things (IoT) and cloud-based architectures [2] is coping with this massive amount of data, mainly for the following reasons. First, limited bandwidth, high processing overhead, and transmission costs make transferring such enormous amounts of data from data sources to the intended destinations highly inefficacious. Secondly, the performance of applications that require real-time processing and response is negatively affected as a result of a considerable end-to-end latency, which is introduced due to the fact that massive amounts of data need to be transferred from end devices to cloud servers that are often not physically close to each other. Finally, some data may entail strict security and privacy requirements that would be violated by having the data transmitted freely over the Internet [3]. Fog computing [4] has emerged as a promising paradigm for tackling the aforementioned issues related to limited network resources, communication costs, and end-to-end latency problems through integrating the advantages of both cloud computing and decentralized processing offered

Kaouther Gasmi and Tahar Ezzedine have contributed equally to this work.

✉ Takwa Allaoui
takwa.allaoui@enit.utm.tn

Kaouther Gasmi
kaouther.gasmi@enit.utm.tn

Tahar Ezzedine
tahar.ezzedine@enit.utm.tn

[1] Communication System Laboratory SYSCOM, National Engineering School of Tunis (ENIT), University of Tunis El Manar, Tunis 1002, Tunisia

Published online: 17 May 2024

by edge devices [5]. This is achieved by allocating computation and storage at the edge of a network rather than utilizing the cloud itself[6].

As fog-based systems ensure additional computing capabilities at the edge of a network, users can profit from the advantages of fog computing mainly by means of computation offloading,[1] which is a mechanism that can overcome the problem of resource constraints at the edge devices. Specifically, it can help improve the performance of computation-intensive applications and battery life [7]. However, selection of the tasks to be offloaded is a challenging problem. Generally, the purpose of decision-making on task offloading is to ascertain whether the offloading is cost-effective for the user equipment in respect of energy consumption and execution delay [8].

Different strategies and techniques have been considered to design efficient computing offloading schemes in the fog environment. The diversity of workloads, task complexity, and QoS requirements of IoT applications in fog environments motivate the use of machine learning (ML) techniques to solve the optimization problem of computation offloading [9, 10]. Machine learning techniques, specifically reinforcement Learning (RL), have been recently widely investigated in the literature to solve computation offloading problems in fog computing environments [11]. This technique is suitable for addressing uncertain and dynamic behavior like fog computing systems. In addition, it can be adopted for decision-making on offloading and enables optimal decisions to be made regarding when offloading tasks is necessary, where to execute them in local or remote resources, what tasks or applications should be offloaded, and how to offload the task. With complex systems and uncertain environments in the real world, the number of devices is generally large; take the example of vehicular networks and Mobile Edge Computing (MEC) [12]. In this scenario, traditional reinforcement learning RL techniques faced challenges when dealing with high-dimensional areas, which require large state space [13]. For this reason, Deep reinforcement learning (DRL) has appeared, integrating RL techniques with deep learning (DL) to achieve high performance in complex and uncertain environments. In addition, using Reinforcement learning (RL) and deep reinforcement learning (DRL) algorithms has been the subject of increasing attention from researchers in the literature. It has appeared as an effective solution to achieve optimal performance when making decisions on offloading in dynamic and complex scenarios.

This paper presents a survey combining RL and DRL techniques to address the computation offloading problem in the fog computing environment. We review the recent studies that address these concerns and investigate different techniques and strategies suggested in the literature.

1.1 Contributions

The main contributions of this paper are as follows:

- We present a comprehensive and detailed survey that reviews current research papers related to the field of computation offloading mechanisms in fog computing using RL and DRL algorithms
- We analyze and classify the recent research papers designed between 2019 and 2023 that propose RL or DRL approaches for computation offloading problems for IoT applications in the context of fog computing.
- We identify a detailed environment, including device size, fog size, and data capture for each use case.
- We propose a classification of RL and DRL-based computation offloading mechanisms in fog computing systems in terms of RL techniques, objectives, architecture, and use cases. Then, in particular, we identify the advantages and weaknesses of each paper.
- Finally, we comprehensively discuss the open issues and future uncovered research challenges to enhance offloading mechanisms in the context of fog computing.

The rest of this article is organized as follows. Section 2 presents related surveys on fog computing. Section 3 presents an overview of the fog computing paradigm: definition, architecture, fog computing architecture, main characteristics, advantages, and issues. Section 4 presents the concept of task offloading and types of offloading. Section 5 presents the basic concepts of Reinforcement Learning and deep reinforcement learning. Section 6 presents the taxonomy of RL and DRL algorithms and classifies recent existing research that employs RL and DRL in fog computing. Section 7 Provides analytical discussion of the reviewed articles. Section 8 discusses open issues and future research directions. Finally, Sect. 9 presents the conclusion of this paper.

1.2 Research methodology

The goal of this study is to systematically review RL and DRL algorithms for task offloading in fog computing. We start by identifying some research questions related to this problem. Then, we describe the research methodology adopted for selecting articles for this survey.

- **Research questions:**

This survey aims to delve deeper into the computation offloading problem in fog computing systems from various aspects, specifically focusing on articles that tackle this

[1] We use computation offloading and task offloading interchangeably throughout this paper, since both terms are used by different studies.

problem using RL or DRL algorithms. We collect and analyze the current articles on computation offloading based on RL and DRL techniques in fog computing and classify the papers based on algorithms and various aspects, to conduct a comprehensive study of the existing literature and identify the open issues. This study addresses five fundamental research questions based on LR and DRL techniques that define the literature review search strategy. The following are the main research questions (RQ) that we are going to answer:

- **RQ1(Category):** What categorization can be assigned to RL or DRL techniques used in computation offloading mechanisms?
- **RQ2(RL techniques):** Which type of RL and DRL techniques are utilized for computation offloading in the fog environment?
- **RQ3(Objectives):** What performance objectives are optimized when solving computation offloading problems?
- **RQ4(Use cases):** What use cases are used for RL and DRL-based offloading, and which environment specifications are considered in each use case, particularly in terms of device size, fog size and data capture?
- **RQ5(Future research directions):** What are the open issues for computation offloading mechanisms, and what are the future research directions?

The answer to this question is crucial for the research community. Sections 6 to 8 provide detailed answers to the above research questions.

- **Source of information**

To ensure the thoroughness of the survey, it is crucial that it covers many potentially pertinent papers. To achieve this, different scientific databases were accessed to extract relevant papers. The selected search databases that have been covered are as follows:

- IEEE Xplore (https://ieeexplore.ieee.org).
- Springer (https://www.springer.com/lncs).
- ScienceDirect-Elsevier (http://www.elsevier.com).
- Google Scholar (https://www.scholar.google.com).
- Wiley InterScience(http://www3.interscience.wiley.com).

- **Search criteria**

We employed a systematic literature review methodology to identify and select the most appropriate papers on computation offloading in fog computing using RL and DRL techniques. Choosing suitable search keywords related to the survey topic is crucial to ensuring that the most relevant studies are selected. Thus, we chose the search terms based on the formulated research questions and combined the keywords with logical operators, notably AND and OR. In the exploration process, the search strings were identified as follows:

[(task offloading) OR (computation offloading) AND (reinforcement learning) OR (deep reinforcement learning) AND (techniques) OR (algorithms) AND (metrics) OR (latency) OR (energy) AND ((Fog computing) AND (IOT)].

- **Inclusion and exclusion criteria**

After searching and identifying related works, the selected papers are evaluated based on both structure and content. To facilitate this process, it is essential to define explicit inclusion and exclusion criteria. The inclusion criteria applied to select the rigorous quality publications are as follows:

– Researches targeting offloading issues in fog environment.
– Papers published in peer-reviewed journals and conferences that have undergone rigorous evaluation procedure.
– Papers published between 2019 and the end of 2023 to provide the most current perspective in this research field.

Concerning the exclusion criteria, the research was eliminated according to the following criteria to exclude irrelevant papers.

– Research not applying RL and DRL methods in offloading mechanism.
– Papers with full text are not available.
– Duplicated papers.
– Books, reviews, and survey papers.
– Research not written in English.

- **Review phases**

This comprehensive literature review explores the most recent papers published in various journals and conferences that have been published during the period from 2019 to the end of 2023, which justifies the novelty and relevance of this topic. To provide a comprehensive study of current research related to the application of RL and DRL in computation offloading in fog computing, Fig. 1 shows the research methodology we followed to select the most suitable papers in this field of research. The following steps do this process: In the first stage, papers are searched in various scientific databases using search keywords.Second, the aforementioned search terms were utilized to check potential papers based on titles, and the irrelevant papers were removed. In the third stage, papers were excluded by

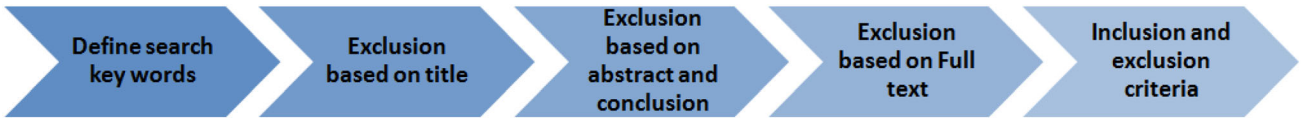

Fig. 1 Process of review methodology

evaluating some critical parts, including the abstract and conclusion. Next, research papers were selected by reading the full-text contents. Finally, papers were selected based on the inclusion and exclusion principle, and the remaining articles related to the RL and DRL-based computation offloading in fog environment were included in the current study for analysis.

2 Existing surveys

Probably the most similar research to the work presented in this article is [14]. In that survey, the authors explore current research proposing RL solutions for computation offloading problems in edge systems. The main strength of the survey is that it classifies the research works according to different aspects, including network architectures, RL algorithms used, decision-making approaches, case studies, and performance metrics. However, the features and weaknesses of each mechanism reviewed were not considered, and most of the included articles were published up to 2020. Recently, and Like the previous approach described above, Abdulazeez et al. [15] conducted a comprehensive systematic literature review (SLR) on task offloading in fog computing using reinforcement learning and deep reinforcement learning algorithms. In this review, they classify the papers according to RL and DRL techniques and divide the article into three categories: policy-based, value-based, and hybrid-based algorithms. As a strength, these classes were compared according to various features, namely problem formulation, RL techniques, performance metrics, tools, case studies, advantages and limits, offloading direction and offloading mode, SDN-based architecture, and offloading decisions. However, in this review, the environment considered by the different articles is not detailed. Firdose et al. [16] also presented a survey related to computation offloading in Edge environment. They focus mainly on the machine learning perspective. They included the three main fields of machine learning: supervised learning, unsupervised learning, and RL. The main feature of this survey is that it provides a comparison between ML-based solutions and the traditional approaches that have been proposed to resolve the computing offloading problem. However, it analyses only 4 articles that use RL algorithms. From another perspective, there are surveys or literature reviews that have focused on the use of RL in resource management, such as the cases in [17–19], where the authors reviewed articles that use machine learning techniques for resource management and computation offloading in Edge computing. Hoa et al. [17] carried out a literature review on RL-based algorithms addressing the resource management problem in the fog computing environment. After introducing the system model, the authors classify the applications of the RL into three different fields: (1) Resource Sharing and Management, (2) Task Scheduling, and (3) Task Offloading and Redistribution. Regarding offloading, the authors introduced the state space used in each proposed solution. Despite the comprehensiveness of this survey, which includes different areas within fog computing where RL is applied, only 5 articles address the issue of computation offloading, and the strengths and weaknesses of each of them are not provided. Muhammad et al. [18] analyze articles that use ML techniques for resource management in Fog computing. The authors classify ML approaches into six areas of resource management: resource provisioning, application placement, scheduling, resource allocation, task offloading, and load balancing. They included the three main fields of machine learning: neural networks (NN), reinforcement learning (RL), and deep learning (DL). The main strength of the survey is that it shows advantages, tools, and performance objectives for each reviewed approach in each area. Sundas et al. [19] extended the previous survey by exploring the use cases for computing offloading in Fog environments such as smart homes, surveillance, smart healthcare, Agriculture, smart transportation, and spacial applications. They examine Centralized vs Decentralized ML Methods and Online vs Offline ML for fog/edge computing. In the same year, Kumari et al. [20] discuss the recent optimization techniques used in computing offloading and the selected metrics. They included three main techniques: Continuous optimization, Lyapunov optimization, RL, and Game theory. The feature of this work is that it provides the mathematical formulation defined for each optimization parameter. Regarding the reviewed RL techniques, the environment that was considered in each work was not detailed. Shakarami et al. [21] investigated a survey on computation offloading mechanisms in Mobile edge computing environments from the perspectives of machine learning techniques. In this case, the selected studies were classified into three main categories: Reinforcement learning algorithm, supervised learning algorithm, and unsupervised learning algorithm to solve the offloading

issue. As an advantage, the survey compares different utilized techniques, performance metrics, case studies, tools, strengths, and drawbacks. However, the surveys did not cover all the reinforcement learning methods.

Although the previous survey presents new perspectives into RL-based computation offloading for fog computing environments, it's important to note that this research field is continuously expanding and evolving by integrating novel RL and DRL models. As a result, there is a strong need for new studies of RL-based computation offloading methods to address new research challenges. Table 1 presents the previous reviews and surveys related to our research, which propose RL or DRL-based solutions for the computation offloading problem. Each survey lists its publication year, technique RL, environment, architecture, resources management and optimization technique. We summarize the weaknesses of most existing surveys as follows:

1. Specific surveys did not consider the strengths and weaknesses of each approach reviewed, which are crucial in classifying research works.
2. The most similar research to our work did not include newly published papers focusing on review articles published up to 2020.
3. None of the existing surveys based on RL has detailed the environment considered in each work.
4. Some surveys did not cover all the RL and DRL techniques.
5. Most of the previous surveys present solutions that do not center their attention on RL and DRL algorithms, considering that extremely few articles have been analyzed using RL and DRL approaches to address the issue of computation offloading.
6. Specific related surveys did not address the issue of computation offloading in the context of fog computing, as the number of papers that used RL and DRL in computation offloading is limited.

The weaknesses and limits discussed previously inspired us to conduct a comprehensive and detailed review of RL and DRL-based computation offloading mechanisms for IoT applications in fog environments. This paper aims to assist researchers in accessing related references. We review the most recent papers and we classify the research works according to different aspects, including RL techniques, objectives, architecture, and use cases, advantages and weaknesses, especially we determine a detailed environment such as device size, fog size and data capture.

Thus, the following are the main contributions of this review:

1. We provide a self-contained overview of the computing offloading problem by discussing some issues that could influence the task offloading decision.
2. We analyze and classify the most recent research papers designed between 2020 and 2023 that propose RL or DRL approaches for computation offloading problems for fog-based IoT applications.
3. We identify a detailed environment, including device size, fog size, data capture for each use case.
4. We propose a classification of RL and DRL-based computation offloading mechanisms in fog computing systems in terms of RL techniques and other aspects, including objectives, architecture, use case advantages and weaknesses.
5. Finally, we discuss the open issues while providing future research directions to solve task offloading problems in the context of fog computing.

3 Overview of fog computing

Fog computing, proposed first by Cisco in 2012 [22], is defined as a distributed computing infrastructure that extends cloud-like services to the network edge by delivering computation and storage resources closer to users [23]. According to the OpenFog Consortium [24],

Table 1 Summary of related surveys

References	Publication year	Technique RL	Environment	Architecture	Resources management	Optimization technique
[14]	2023	✓		✓		
[15]	2023	✓		✓		
[16]	2021	✓				✓
[17]	2022	✓			✓	
[18]	2022	✓			✓	
[19]	2023	✓			✓	
[20]	2022	✓				✓
[21]	2020	✓				✓
Proposed	2024	✓	✓	✓		✓

Fog Computing is a horizontal, system-level architecture that distributes computing, storage, control and networking functions closer to the users along a Cloud-to-Thing continuum.

As opposed to cloud computing, fog computing introduces fog servers and fog nodes, which are devices physically closer to the users than their cloud counterparts, with a goal of handling some of the application workload closer to the network edge. Any device such as a controller, smart gateway, switch, router, or embedded server, which possesses capabilities for processing, networking and storage may be employed as a fog node. The advantage of these devices is wider deployment opportunities, as they can be placed at any location where network connection is available, such as inside factory buildings, alongside railways, inside of vehicles, and even on power poles. The main idea behind this approach is to optimize the transmission time it takes for data to reach the intended processing nodes, since deploying nodes closer to the edge of a network shortens the data transmission time to a negligible delay [23]. According to OpenFog, fog nodes are equipped with intelligent algorithms that facilitate processing and storing data, as well as forwarding data from edge devices to fog, and from fog to cloud via smart networking.

3.1 Fog architecture

A number of architectures for fog computing have been proposed, most of which have a three-layer structure as illustrated in Fig. 2. A typical fog architecture comprises an IoT layer (also known as end layer) that consists of IoT devices, a fog layer that consists of fog nodes, and a cloud layer with a cloud data center and services [25], which are discussed in more detail below.

- IoT layer: The IOT layer is situated at the bottom and closest to end-users. It tier consists of all the IOT devices connected to the internet, including sensors, actuators, smart vehicles, smartphones, connected cars, etc. [26]. In addition, the computation resources of IoT devices are limited; they have low computational power, are constrained by battery life, and have low processing capacity. These IoT devices generate enormous amounts of data and send it to the upper layer for processing [27].
- Fog layer: It is an intermediary layer located between a cloud and IoT devices [26], which consists of heterogeneous fog devices that have limited processing, storage, and networking capacity Known as Fog Nodes [28], these devices include access points, base stations, routers, gateways, switches, etc [29]. These fog nodes maintain a connection to cloud servers and can forward requests to cloud data centers. The fog node can be high or low level. Fog nodes with very limited processing and storage are considered low-level nodes, while nodes with rich resources are classified as high-level nodes [30, 25, 31].
- Cloud layer: The top-most layer of the edge-fog computing architecture is known as the Cloud layer. It includes various cloud servers and cloud data centers with significantly more computation power, efficient storage, and processing capabilities than the fog layer. This layer can perform complex processing and store large amounts of data [26, 25, 31].

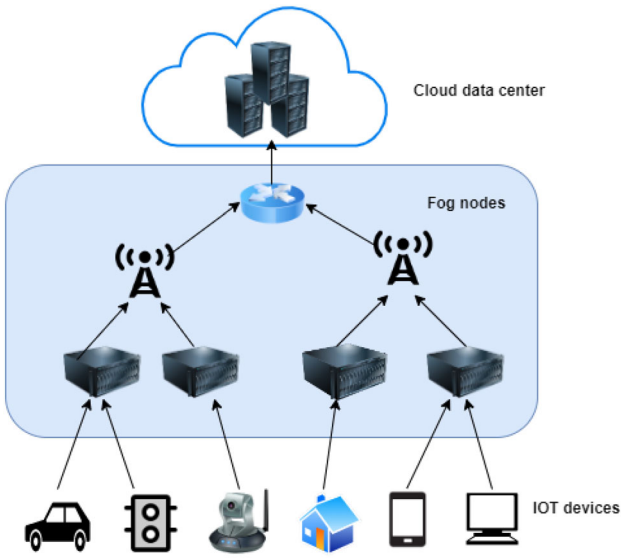

Fig. 2 Illustration of fog computing architecture

Considering this layered fog architecture, it is evident that fog services are located in a much closer proximity to the end-users, allowing a denser geographical distribution, and offering a better mobility support, as opposed to a cloud environment in which services may reside in much farther locations necessitating wider bandwidths. Furthermore, a fog ecosystem also contributes to reduced latency and provides context awareness due to fog node localization, resulting in pervasive, scalable, and united network connectivity [6].

3.2 Fog computing architecture

Fog computing offers a versatile architecture that can be adapted to the specific requirements of the application. We identify three distinct types of architecture based on where the agents are located: distributed, centralized, or decentralized.

- Distributed: In the distributed architecture, several agents are located on fog servers, which are responsible for making offloading decisions for user devices

connected to them [14].The multiple fog servers communicate and synchronize with each other to present themselves as a single system. Distributed systems generally operate in two modes: either a separate fog node contributes to achieving a common shared goal, and the user device receives the outcome as a coherent entity. Or each node serves its own user device, and the distributed architecture facilitates resource management. In this architecture, the network controller distributes tasks to servers according to different factors, such as latency and resource requirements [32, 33].

- Centralized: In the centralized architecture, the system has a central server with high computing capability, which is responsible for offloading decisions for all tasks generated from the end devices.When a task needs to be executed, the client sends a task request to the central server, which then processes the task and sends the results. While, centralized architecture exhibits lower efficiency and an increased susceptibility to failure than distributed and decentralized architecture. It offers more security for user data since it's stored in one location [32, 14].
- Decentralized: In decentralized architecture, each server makes its own decisions, which means that decisions are made at different levels. The overall operation of the system is defined as the coordinated decisions made collectively by the participating servers, guaranteeing highly fault-tolerant and without a single point of failure. Implementing such systems poses significant challenges, particularly in managing replicas [32].

3.3 Key characteristics and advantages of fog computing

The distinct characteristics and advantages of fog computing can be listed as follows.

- Low latency: The key motivation behind this emerging paradigm is to decrease the data transmission latency while increasing the data transmission rate [34]. Some novel applications such as intelligent vehicle-to-vehicle communication networks, virtual reality, and online gaming have an extremely high requirement for low delay. For instance, in a virtual reality application, a small delay in range of milliseconds can damage the user experience [35]. Due to the close proximity of fog nodes to end users, fog computing is capable of providing support for low latency and time-sensitive applications [36].
- Support for mobility: In applications such as vehicular networks the movement of the nodes can considerably affect the system performance, especially in scenarios that require handling fast channel changes [36]. Fog computing can facilitate the mobility of the end users through providing computational and storage resources over the entire network, as opposed to a traditional centralized cloud ecosystem [25].
- Bandwidth: Preprocessing of data before transferring it to a cloud for further analysis or storage plays an important role in reducing the network traffic. Fog computing enables data filtering and aggregation to be performed locally in order to speed up the execution of certain tasks that would otherwise take too long under a limited network bandwidth [25].
- Scalability: There are IoT scenarios in which a huge number of end-users need to be managed along with enormous amounts of data produced by billions of heterogeneous IoT objects with different performances and costs. In such scenarios, having fog nodes widely distributed and deployed in close proximity to the users plays an important role in making IoT applications scalable and adaptable to the network changes [36].

3.4 Fog computing issues

Fog computing paradigm promises to offer effective solutions for a number of problems that afflict IoT and cloud computing applications. Nevertheless, it also comes with its own challenges that need to be grasped and tackled in order for it to become an effective computing solution. Some of the fog-specific issues are shown in Fig. 3.

- Resource management: Fog environment contains heterogeneous fog nodes with limited computing and storage performance capabilities, which makes it harder to manage resource allocation, scheduling, and sharing. Efficient management solutions are needed for determining appropriate task placement strategies that satisfy applications' requirements (e.g., solutions based on priority and migration) [37]. Furthermore, in order to provide mobility support, particularly in the case of connected objects, resources may need to be preallocated using effective methods such as probabilistic ones based on user history [38]. Therefore, a framework that enables the performance evaluation of resource management policies in IoT or fog computing infrastructures is a necessity [39, 6].
- Privacy and security: Some of the proposed solutions for security issues in fog computing focus on intrusion detection [40], access control [41], authentication [42], and detection of various other malicious activities including denial of service (DoS) attacks and port scanning [43]. Introducing security mechanisms in fog

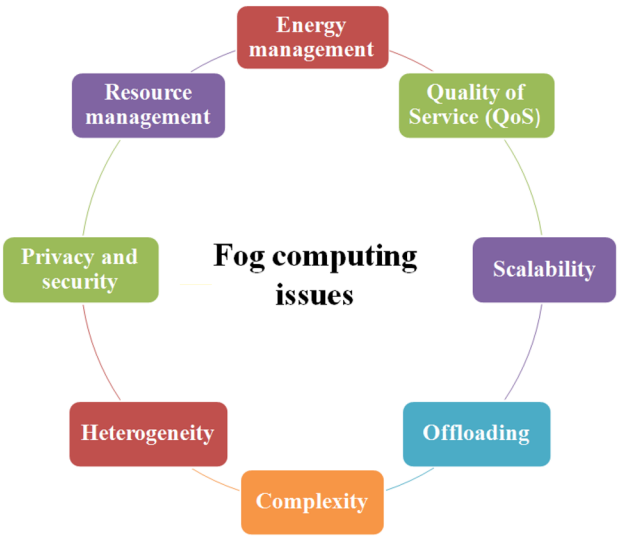

Fig. 3 Fog computing issues

computing is necessary for each level of the fog architecture [44].

- Energy management: A fog infrastructure incorporates a large number of geographically distributed nodes. As a result, energy consumption in a fog ecosystem is higher in comparison with that of a cloud [35, 45]. Significant research efforts are needed to develop effective solutions for energy management. For example, data processing and communication protocols that are less costly in terms of energy consumption need to be developed [45].
- Quality of Service (QoS): One of the most important measures of network service quality in general is QoS, and this holds for fog computing as well. In [46], the authors consider QoS in fog computing based on four metrics: connectivity, reliability, capacity, and latency. Other metrics such as energy efficiency, bandwidth, and security may be taken into consideration as well, depending on the application requirements. The challenge with QoS provisioning often entails making trade-offs between different QoS metrics, which is further complicated by a varying and often dynamic nature of different applications.
- Offloading: Offloading in a fog computing environment refers to transferring intensive computation tasks from resource-constrained devices to nearby resource-rich devices such as those used in Fog and Edge Computing[16]. The problem of task offloading in fog computing is deciding which task to offload for each end-user device, where, and how to offload it [18]. The main objectives of task offloading are to support the resources constrained of IoT devices and enhance the system's effectiveness by reducing the total delay and energy consumption of devices. In section 4, a more detailed definition of task offloading is introduced, and task offloading types are presented [15].
- Scalability: The proliferation of IoT devices results in the generation of massive volumes of data demanding huge amounts of resource-rich devices with storage capacity and processing power. Consequently, the fog system should be capable of managing all the end-user devices with appropriate resources. The main challenge of scalability is the capability of the fog server to react according to the rapid growth of end-user devices in order to deliver services successfully [47, 48].
- Heterogeneity: Fog computing is, by nature, heterogeneous, whether at fog nodes or network infrastructure. Countless IoT devices, such as sensors, mobile phones, and vehicles, contribute to increasing the heterogeneity of the fog computing environment. These end devices have different capabilities, including computing powers, communication radios, and storage [48]. The challenge will be managing these heterogeneous networks of IoT devices and selecting suitable fog resources [49, 50].
- Complexity: The complexity issue in fog computing is mainly due to the complex nature of managing a widely diverse environment. Considering the multiple IoT devices designed by different manufacturers, the complexity of the fog nodes increased. Moreover, various software and hardware configurations intensify this complexity. Thus, Choosing the most suitable product to enhance performance has become increasingly challenging [48].

4 Task offloading

In heterogeneous and complex environments, fog computing is frequently used. The computing capabilities of the devices located close to the edge of the network are insufficient to support the heterogeneous modules of various applications in the IoT ecosystem, so the strategy must be formulated to keep these constraints in mind. There is no doubt that further research will be required to address the challenges associated with the evolution of the Fog-Cloud architecture, and among these challenges is the efficient deployment of offloaded applications. Thus, the question that arises is, what tasks should be offloaded? How to offload it? And where should your tasks be offloaded to? Several researchers have addressed these questions, and many strategies have been proposed accordingly for efficient computation offloading.

Task offloading is a technique by which an IoT device transmits some or all of the local tasks to a neighboring fog node or cloud for execution in order to alleviate the

workload of the IoT device and improve system performance. IoT devices are usually resource-constrained and task offloading allows these devices to transmit the intensive tasks to the fog servers and subsequently get back the computation result in return [51]. The transmission procedure is referred to as offloading, and the portion of offloaded tasks is referred to as offloading fraction. Generally, the offloading process involves a number of constraints that must be satisfied in order to optimize offloading objectives. These constraints include many factors, such as limits on delay, energy consumption, and maximum bandwidth utilization [52, 53]. Therefore, offloading intensive tasks from IoT devices to fog nodes reduces the execution time, supports real-time applications and extends the battery life of IoT devices[54]. We present various types of task offloading depending on the task-splitting decision taken. Figure 4 illustrates the result of that decision. In this illustration, we split the tasks generated from tablets, smartphones, and VR glasses into six sub-computations (C1-C6). Subsequently, we determine which sub-computations should be offloaded remotely and which ones are executed locally, considering delay and energy considerations. The classifications of computation offloading decisions are given below:

- Local execution: The local execution refers to the process of running tasks directly on the IoT device. The whole computation is performed locally at the end device without relying on external resources(cloud or fog servers). Some tasks require local computation because they are very sensitive to latency[51], this process is suitable in scenarios where devices have sufficient resources to perform the task efficiently, when offloading is not cost-effective or when fog resources are not accessible. In Fig. 4, the tablet performs all computations locally [55].
- Partial offloading: Partial offloading allows a task to be partitioned, a portion of the computation task is executed locally, and the remaining is offloaded to fog nodes or cloud server for execution, [51] Partial offloading is also called dynamic offloading or fine-grained offloading. In this offloading type, collaborative computing is established between cloud and fog node resources to execute the offloaded tasks from end users. The advantage of this collaboration is that we can benefit from local and remote resources, particularly in large-scale scenarios where the fog resources are insufficient to process all the offloaded tasks. However, the main challenge in such a method is task offloading decisions and task scheduling, considering the end device's resource constraints and energy consumption. The first decision-making level is at the local device, which decides which tasks will be migrated to a remote server. Significantly, the local device must determine which tasks can be executed locally and which are to be offloaded to fog or the cloud. Once the local device has decided which tasks are to be offloaded, the next decision-making level occurs. At this point, the fog infrastructure performs a second task, partitioning. The aim is to decide which subset of tasks should be executed in the fog resources and those that should be sent for execution in the cloud. On the other hand, when tasks are transferred to the cloud, delay constraints and network reliability become critical factors and must be taken into account, as they can limit the overall system performance [54]. In Fig. 4 the VR glasses perform four computations tasks locally and offload the other two to the fog server.
- Full offloading: The whole computation is offloaded from a local device to a remote server to be executed. Full offloading refers to the process of relocating all tasks to the cloud or fog resources for computation, it is also called total offloading or coarse-grained [51]. This approach is simple for implementation and very suitable when tasks cannot be divided. While this procedure offers substantial energy savings for IOT devices, it is important to take into account the other sources of energy dissipation, such as the device's transmission power[54]. It should be noted that these methods do not concern the decision-making process for task offloading in fog cloud computing because the user is responsible for the choice of tasks to be offloaded[53]. In the case of the smartphone shown in Fig. 4, all computations are offloaded to the fog server.

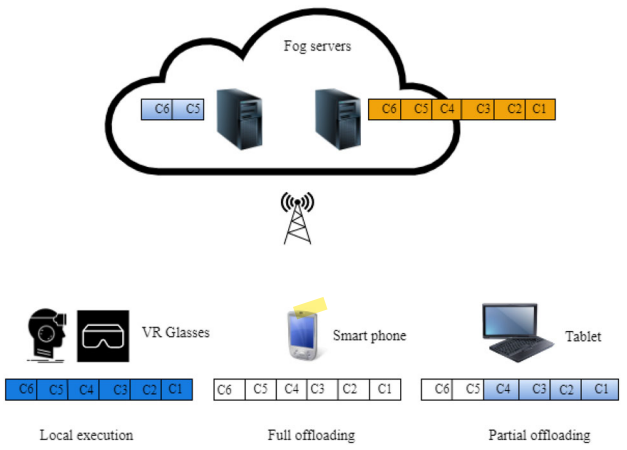

Fig. 4 Outputs of computing offloading decision for different application components at mobile devices

5 Reinforcement learning

Different methods have been designed to handle the issue of computation offloading in fog computing environments. These approaches include traditional methods like on nonlinear programming [56], mixed integer programming

[57], and heuristics algorithms [58]. Although this traditional algorithms can find a solution in simple scenarios, they are generally unable to solve problems efficiently when confronted with complex and dynamic environments. Therefore, these approaches have their limitations, as task offloading and resource management are carried out without consideration for the diversity and dynamics of workloads [59]. Fog computing systems generally involve many devices and fog nodes, which communicate with each other and generate tasks dynamically with specific resources and latency requirements, resulting in increasingly diversified and dynamic workloads that change with time. However, most of the traditional approaches are static and configured offline for specific workloads. They can't respond dynamically to changes and adjust the scaling of applications according to workload [60]. Therefore, resource management is a significant problem, and it's not straightforward to decide which tasks should be offloaded and where. Moreover, in such a scenario, there is a need for making real-time decisions, and incorrect decisions when the system is scaled up can cause performance deterioration [61]. The task complexity of IoT applications serves as motivation to use machine learning (ML) techniques to address the computation offloading problem. In addition, ML techniques, especially Reinforcement Learning (RL), can be utilized for offloading decision-making and resource management to reduce workload in large-scale systems [62]. RL is a branch of ML that enables learning how to make a decision through interaction with the environment. This makes RL a suitable technique for addressing dynamic environment. In other words, RL techniques allow the best decisions to determine how, when, and where to offload tasks [63, 19, 15]. In the following section, we present the basics concepts of reinforcement learning.

5.1 Basics concepts

Reinforcement learning is a machine learning technique that is neither categorised as supervised nor unsupervised. Reinforcement learning is a paradigm that differs fundamentally from supervised learning which trains a model to learn from labelled data, similarly it differs from unsupervised learning which does not need labelled data to learn. RL is a completely different method and categorized as a third paradigm because it discover and learn for itself based on rewards and punishment. One of the definitions given by the authors is that reinforcement learning is a decision making process used to solve issues in a large range of domains (Sutton and Barto,2018) [64]. In other words, reinforcement learning is the conception of a relational agent that acts with intelligent behaviour like a human (Sewak,2019) [65]. Moreover, it is an autonomous learning process that allows an agent to learn from experience in order to maximise the cumulative reward based on trial or error.

To address decision making problems in Reinforcement Learning environment Markov decision process MDP is often used. MDP is a mathematical method composed of a set of states, actions, transitions between states and rewards (Puterman, 2014)[66]. Figure 5 illustrates a basic design and the interactions of reinforcement learning system in an MDP. At a time t, the decision maker or agent interacts with the environment to choose the optimal action (A_t) in the present state (S_t) of the environment. When the agent takes an action then the state of the environment changes from (S_t) to (S_{t+1}) and gives a reward (R_t) for the agent. Then the agent takes the best action for the new state (S_{t+1}), which subsequently generates a reward (R_{t+1}). The following paragraphs define the basic concepts and notations used to describe the RL algorithm [65].

Considering that we will use RL and DRL technique in our research, we will first define the terms and notations of RL system to better understand how the system works. Then, we will present in the following subsection deep reinforcement learning

- **Agent**: Agent is a system or robot which interact with the environment and makes intelligent decision on the action to be taken using observations and the current state of the environment In artificial intelligence, RL agents are regarded as the most developed and able to perform at advanced levels of intelligence[65, 67].
- **Environment**: The environment is defined as the external world in which the agent interacts. The role of the environment is to give the agent the different states that exist and according to it the RL agent make the appropriate decision [65]. The environment can have several states, it changes while the agent is interacting with it or it can change by itself [67].
- **State space**: A state S is a description of the existing environment or world at current situation or time, it is a unique description of the important elements of a state of the modelled problem (Van Otterlo-Wiering,2012) [68]. The state space is defined as the set of states S = {s1, s2,.., sn} available to an agent in the environment

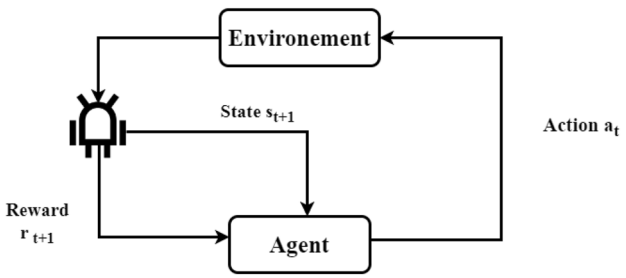

Fig. 5 Design of Reinforcement learning system

set of states → available to an agent → in the environment at any discrete time

at any discrete time. In reinforcement learning, experiences and interaction with the environment trains and the agent through trial-error the correspondence between the situation and actions in an autonomous way. Immersed in the environment, the LR agent observe the set of information in the environment that affects the action to take by the RL system. In this way, the states impact on the action taken by the agent [67].

- **Action space**: An action is the manner in which an RL agent reacts with the environment. Environment influences actions, thus a different environment gives different actions. Furthermore, the state of the system can be controlled by actions (Puterman,2014)[66]. Action space can be defined as the set of actions that the RL agent might take in the environment. Actions are divided into two types: discrete and continuous. An environment is said discrete when it has discrete action spaces for example: Games, Atari, Go [69]. While continuous environments have continuous action spaces, taking the example of an agent manages a physical robot in real world [70, 67].
- **Reward**: A reward is used to identify the way in which the MDP system should be controlled (Van Otterlo-Wiering,2012) [68]. To simplify the learning process, when an agent makes an action, it receives either a reward or a penalty for each tentative depending on the action taken in a given state. This means, a reward is a function of actions and states, is denoted R: S×A×S [65]. In RL, a reward is represented by a positive or negative numerical value to control the agent's behaviour and determine the desirability of the action. The objective of agents is to maximise the total reward which includes all the available rewards that the environment can provide [67].

$$R_t = \sum_{k=0}^{\infty} r_t + k + 1 \tag{1}$$

- **Policy**: A policy is the set of rules that the learning system uses to determine the optimal action which an agent can take based on the current state, is denoted by π. The goal of RL algorithms is to generate an optimal policy to maximize system's performance by determining the appropriate action to take in a specific state. In Reinforcement Learning there are two approaches used to training an agent: on-policy method and Off-policy method [65].
- **State Value and State-action Value Function**: The value function or state value function is noted $V^{\pi}(S)$ where (S) means that this value represents a function of state. Value functions estimate how good it is for an agent to perform an action in a given state. The rewards that the agent can expect to obtain in the future are related to the actions that will make [64]. Thus, the state value function $V^{\pi}(S)$ is defined by the reward expected by the agent starting from a particular state then following a defined policy (Otterlo-Wiering,2012)[68]. The mathematical formulation of state value function is expressed as:

$$V^{\pi}(S) = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k r_t + k + 1 | S_t = S] \tag{2}$$

The state-action value or Q-function is a function that depends on the state and the action denoted by Q(s, a). Thus, the Q-function is used to evaluate how good it is to take a particular action by the agent in a given state according to a specific policy [65]. The Q-function can be formulated as:

$$Q^{\pi}(S, a) = \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k r_t + k + 1 | S_t = S, a_t = a] \tag{3}$$

In the context of large state and action spaces of large-scale networks and complex scenarios, RL techniques face difficulties in finding the optimal strategies within the given deadline. Reinforcement learning is not well adapted to high-dimensional spaces. Hence, deep reinforcement learning (DRL) emerged, which combines RL with deep learning (DL), delivering high performance in complex and dynamic environments[15].

5.2 Deep Reinforcement Learning

Despite the successful use of RL in a wide range of applications, it poses many challenges, especially when it comes to solving real-world problems [71] where a considerable number of states or actions, storage, and computation become incredibly high [72].

In addition, tabular representations, as a way of storing values for state or state-action [73], where each state-action value is assigned a discrete estimate, are not adapted to the large scale of state and action spaces. Furthermore, given the diversity and uncertainty of system components, there may be infinite system states. In such instances, the computation cost of all Q-values becomes unfeasible. This problem is known as the "curse of dimensionality." This term was invented by Bellman, which can result in an unsolvable problem in real applications due to storage and computation constraints[74].

To address this problem, deep reinforcement learning (DRL) uses Deep Neural Network (DNN) instead of tabular representations to overcome the limitations of reinforcement learning: $Q(s, a, \theta) \approx Q_{\pi}(s, a)$, where θ is the Neural Network parameter and $Q(s, a, \theta)$ is the Neural Network output that is roughly equivalent to $Q_{\pi}(s, a)$. Deep reinforcement learning (DRL) is a subclass of

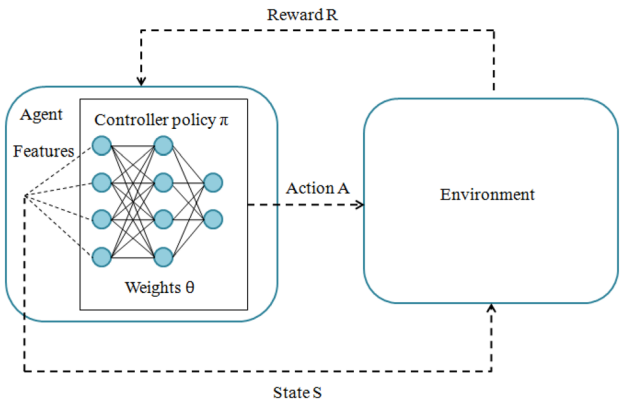

Fig. 6 Working of deep reinforcement learning

Reinforcement learning methods that combines deep learning (DL) with reinforcement learning (RL) to learn an agent to make decisions in complex and uncertain environments [75]. This technique exploits the advantage of DNN to estimate the value and policy learned by storing state-action pair values in RL, thus enabling faster learning and improving the performance of reinforcement learning algorithms [76]. The DRL is illustrated in Fig. 6. In addition, DRL encompasses a variety of techniques, including Policy Gradient (PG), Deep Q-Networks (DQN), Double Deep Q-Networks (DDQN), Deep Recurrent Q-Networks (DRQN), Deep Deterministic Policy Gradient (DDPG), Dueling DQN, and other techniques such as Actor-Critic methods (A2C), (A3C), and Soft Actor-Critic (SAC). These techniques are discussed in more details in the following section.

6 RL based task offloading mechanisms

6.1 Taxonomy of RL and DRL based algorithms

RL algorithms can be divided into two classes: model-free and model-based. In a model-based algorithm, the agent generates a detailed model of the environment. Generally, this model contains a representation of the state space, action space, and reward function, and it learns the optimal policy from this information. On the other hand, in Model-free RL, the model is not known. It learns the optimal policy from interaction with the environment. This approach aims to initiate an arbitrary policy, estimate the value function, and then update the policy based on the rewards received. This review is focused on model-free methods to address offloading problems in a fog environment. There are three main sub-types of model-free, including policy-based methods, value-based methods, and hybrid methods.

The first is policy-based, which optimizes the parameters θ of the policy without relying on value function estimation. This approach explicitly constructs a policy that is a mapping between states and actions using different techniques such as A2C/A3C, in which performance is maximized using gradient ascent, and the other is proximal policy optimization (PPO).

The second method is Value-based, the value function is used to evaluate states based on the cumulative reward that the agent gets with time. Value-based approach like temporal difference (TD) is used to estimate the value function instead of explicitly learning the policy. Such value-based algorithms aim to estimate the optimal value function, which is subsequently employed to derive the optimal policy. Standard value-based RL algorithms, include SARSA, Q-learning, Deep Q-Networks (DQN), Double DQN, DRQN and Dueling DQN. Sarsa refers to "State-Action-Reward-State-Action" is categorized as an "on-policy" method, meaning that it updates and adjusts the same policy used for making decisions [64]. Unlike SARSA, Q-learning is an off-policy method, indicating that it evaluates and updates a different policy from the one used to make decisions. Q-learning and SARSA have been implemented as tabular methods, which store the value of all possible state-action pairs. For Q-learning, the Q-table is randomly initialized, and the process of updating requires the current value $Q(s, a)$ with the maximum value of $\max_{a'}$ Q(s',a'). Hence, the value corresponding to all state-action combinations is stored in a table, where $s \in S, a \in A$. The Q-learning approach is an efficient way of achieving an optimal policy when the number of actions and space are small. However, with complex systems and uncertain environments which require large state space this technique faced challenges [14]. For this reason, the Q-Learning method is infeasible and can fail to find the optimal policy. Thus, the Deep Network (DQN) was introduced by Mnih et al. in 2015 [77] to address this problem. The deep Q-Network (DQN) approach is based on the concept of a neural network that adjusts Q-functions to estimate the optimal value of Q(s, a) [78]. In the same way as the Q function in RL, the DQN approach takes the current states as input, and then all the input states are transferred to the layers of the neural network through particular weights and return a set of Q values for each action in that state $f : st \rightarrow (Q(s_t, a_0), .., Q(s_t, a_n))$ [75, 79]. In addition, DQN is the most popular DRL algorithm that integrates DL and Q-learning to address computation offloading problems. Many extensions were developed to enhance the performance of DQN. These include Double DQN (DDQN), Dueling DQN, and Deep Recurrent Q-Network (DRQN).

The last category is Hybrid algorithms RL, which combines value-based and policy-based approaches to

benefit from the advantages and strengths of each algorithm. These hybrid approaches are designed to implement policy-based methods for modeling the policy function while relying on value-based algorithms to update these policy functions. The DDPG and SAC are two examples of hybrid RL algorithms [15].

This research paper investigates model-free methods to tackle computation offloading problems in a fog environment. Figure 7 illustrates a taxonomy categorizing RL and DRL algorithms into three main classes: value-based, policy-based, and hybrid-based algorithms. Below, Table 2 presents, in their "RL technique" column, the algorithms used in the reviewed papers to solve the issues of computation offloading. In addition, RL and DRL-based computation offloading mechanisms can optimize system performance such as latency, energy consumption and QoS.

In the literature, various studies based on Reinforcement learning (RL) and Deep reinforcement learning (DRL) have discussed task offloading problems in fog computing and have been performed recently for efficient task offloading. In this review, we classify the state-of-the-art studies based on RL techniques which are organized according to three classes: value-based, policy-based, and hybrid algorithms. Table 2, provides a summary of the surveyed studies:

6.1.1 Policy based

6.1.1.1 A2C/A3C Many offloading problems have been solved using actor critic methods A2C/A3C, for example

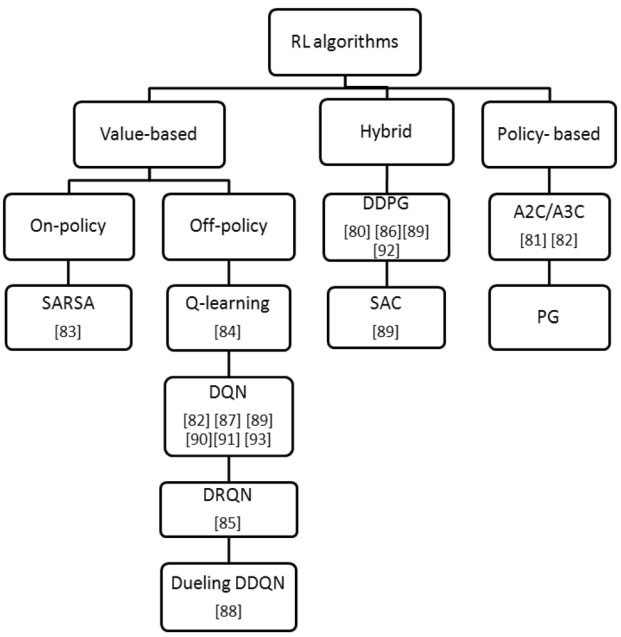

Fig. 7 Taxonomy of RL algorithms

authors in reference [81], explored the multi-agent approach to solve the issues of computation offloading and resource allocation. In this contribution, they proposed a deep reinforcement learning (DRL) based on the actor-critical advantage (A2C) approach in the IoT-Fog-Cloud framework. According to this strategy, they implemented a single-agent reinforcement learning model immediately to a multi-agent to offload the tasks into a single fog node for processing. The main objective of this paper is to jointly minimize the offloading latency and optimize the resource allocation by coordinating resources of offloading tasks so the task can be processed locally, offloaded to the fog layer to reduce latency, or executed to the cloud. Their simulations were done on a network comprising up to 100 fog nodes and over 100 devices in general applications. Based on the analysis done on this approach,the simulation results revealed significant reductions in latency from 1.1s to 0.2s compared to other techniques, and they found a considerable 80% less cost in terms of latency. Hence, this technique effectively enables task offloading in a short time, meeting the latency requirements for efficient fog computing applications. However, the proposed offloading system should also take into account energy consumption issues. Although the numerical simulations proved that A2C could minimize the latency, this scheme is not scalable, considering that when the number of IOT devices increased and the number of fog nodes kept constant, the latency cost increased. Another negative point of this paper is that the proposed approach is not compared with other similar algorithms. Therefore, comparison with similar algorithms is essential to identifying the competitiveness and strengths of the scheme.

Authors in reference [82] investigated a framework known as a deep reinforcement learning-based cloud-edge collaborative mobile computation offloading (DRL-CCMCO) approach using the concepts of DQN and A3C to solve offloading decision and resource allocation problems in edge-cloud computing. In this framework, the network is composed of a mobile edge computing offloading model with three layers of digital twins and a decentralized network of computation resources to support the mobility of user terminals. The objective of the optimization problem in this contribution is to minimize the weighted sum of the total delay and energy consumption. The simulations were conducted on a network consisting of up to 100 fog nodes and close to 100 devices in industrial applications. Based on the analysis done on this approach, when the number of edge nodes increased, the computing capability of the system also increased. Thus, the execution time was reduced by 50%,15%, and 12% compared to deep Q network (DQN), deep deterministic policy gradient (DDPG), and A3C algorithms. In this way, the CCMCO algorithm can quickly and dynamically make the optimal offloading

Table 2 Task offloading strategies

Ref	Technique RL	Objective	Paradigm	Architecture	Offloading type	Application	Environment			Advantages	weaknesses
							Data capture	Devices size	Fog size		
[80]	DRL DDPG	QoE	MEC	Distributed	Full	Mobile applications	Real	> 1000	> 1000	High system utility	-Scalability
[81]	DRL-A2C	Latency	Fog	Distributed	Partial	General applications	Synthetic	> 100	< 100	Minimize latency and system cost	-Scalability -Energy consumption is not considered -Approach not compared with other algorithms
[82]	DRL-DQN A3C	-Latency -Energy	Edge	Distributed	Full	Industrial applications	Synthetic	< 100	< 100	-Fast convergence speed -High stability -Minimize the total cost of execution delay and energy consumption	-Scalability is not guaranteed in large-scale areas -Heterogeneity of edge-cloud is neglected
[83]	RL-SARSA	Energy	Fog	Distributed	Full	Smart villages	Real	N	< 100	-Minimize energy of RSU with a tolerable response latency -Fast convergence	-No security -The proposed system is not compared with RL algorithms
[84]	Q-learning	-Latency -Energy -CPU	Edge	Distributed	Partial	Mobile applications	Real Synthetic	< 100	< 100	Minimise energy consumption, execution time and CPU utilization	-No security -RL may face challenges in complex scenarios
[85]	DRL-DRQN	Latency	Fog	Distributed	Partial	General applications	Real	N	< 100	Minimizing delay	The proposed approach is based on SDN architecture which has drawbacks such as security, scalability and reliability
[86]	DRL DDPG	Energy	Fog	Distributed	Full	General applications	Synthetic	< 100	< 100	Reduce energy consumption	No security and privacy
[87]	DRL DQN	Latency	MEC	Centralized	Full	Mobile applications	Synthetic	< 100	< 100	Decreased the computation time	-No scalability -When the action spaces grow single agent can't solve well the problem of large action spaces
[88]	DRL-Dueling DDQN	Latency	MEC	Centralized	Full	Mobile applications	Synthetic	N	1	Reduce long-term overall system delay	-Energy consumption is not considered an important issue -Approach not validated in a real world situation
[89]	DRL DQN SAC DDPG	QoS	Fog	Distributed	Full	General applications	Real	< 100	< 100	DRL can satisfy the deadlines and QoS requirements	-No mobility -Approach not validated in a real world environment

Table 2 (continued)

Ref	Technique RL	Objective	Paradigm	Architecture	Offloading type	Application	Environment			Advantages	weaknesses
							Data capture	Devices size	Fog size		
[90]	DRL DQN	QoS	Fog	Distributed	Partial	Vehicular application	Synthetic	> 100	> 100	Improve task offloading and resource allocation performance	-No security -Mobility not considered -Approach not compared with RL techniques
[91]	DRL-DQN	QoS	Edge	Distributed	Partial	Industrial applications	Real	N	> 1000	-Enhance the QoS -Privacy guarantee	-Approach not compared with RL techniques
[92]	DRL DDPG	QoE	Edge	Distributed	Full	General applications	Synthetic	> 100	< 100	-Improve QoE -Lower variation and higher rewards	-Approach not validated in a real scenario -The scalability is not discussed for large-scale areas
[93]	DRL DQN	Energy	Edge	Distributed	Full	Mobile applications	Synthetic	N	< 100	-Fast convergence -Minimize energy consumption	-Approach not validated in a real scenario -Latency is not considered an essential problem

decision. Moreover, the simulation results showed that the suggested approach decreased significantly the total cost from 25% to 60%. These outcomes indicate the ability of this strategy to reduce latency while using less energy. Overall, this system provides a balanced trade-off between latency and energy consumption. Although these schemes can be used to find the optimal offloading decision, a small-scale area is discussed in this work, and the resource heterogeneity between the edge and cloud is neglected.

6.1.2 Value based

6.1.2.1 SARSA SARSA has also been one of the RL techniques that researchers have addressed to solve the computation offloading problems in fog systems. For example, in [83], the authors studied the computation offloading problem in a vehicular network. To address this problem, they proposed a fuzzy reinforcement learning (FRL) method for vehicle scheduling in vehicular networks to solve the problem of computation offloading and task scheduling produced by IOT applications operating in smart villages. The main goal of this study is to minimize the energy consumption of the Road Side Units (RSU). The authors combine an on-policy reinforcement learning technique called SARSA and a fuzzy logic-based heuristic as a solution. Simulations are carried out on an environment comprising up to 100 fog nodes and several devices in vehicular network application, and has been tested using a dataset of input vehicular traces for the highway environment collected from researchers. Then, the results obtained are compared with other different methods. The analytical results showed that the proposed approach could significantly reduce energy consumption compared to both the DTA DP and Fuzzy methods, with reductions of 48.27% and 51.61%, respectively. However, FRL does not achieve better latency results; it is noteworthy that the total service time of the FRL approach is 10 s higher than that of the other alternatives considered. In this way, FRL excels at optimizing energy consumption but falls short when it comes to latency because it exhibits sub-optimal latency performance compared to the other approaches. The observed increase in the total service time with this contribution suggests that it takes more time to achieve this objective, which can lead to higher service time. Overall, this approach achieved strong results in terms of energy consumption, and it is especially adapted for applications that prioritize energy efficiency rather than latency. However, in latency-sensitive applications, the service time issues may restrict the system's overall performance. As another drawback, the authors only compared the proposed method with non-RL algorithms. Furthermore, security issues that can impact network performance are neglected in this work, and it is recommended that security technology be integrated into the proposed approach to enhance the system's robustness.

6.1.2.2 Q-learning The Q-learning method was introduced to address RL-based computation problems in fog computing. For example, in this paper [84], authors mainly discussed the challenges related to computing offloading. This contribution is typically based on the RL technique in which authors proposed an hybrid model which combines LSTM with Q-learning algorithm. They aim to minimize latency, energy consumption and CPU utilization. The proposed solution is validated in mobile applications using simulations with synthetic and Chicago taxi trip dataset as real-world workload traces, and the network size includes up to 100 fog nodes and up to 100 devices. Simulations results showed that combining RL and LSTM techniques outperformed the other algorithms in minimizing execution time, energy consumption, and CPU utilization. According to analytical results,the proposed approach reduces the execution time by up to 8.3% and 11.3% compared with other approach it was able to reach the the main objective in a reasonable amount of time. Moreover, the simulation results revealed significant reductions in energy consumption from 8.4J to 31J compared with other approaches. Overall, this approach achieved a significant reduction of execution time and offers a balanced compromise between latency and power consumption. Despite this approach improved the capabilities of the edge computing environment. It has some limits, because this work does not consider security and privacy.

6.1.2.3 DQN Authors in [82] studied the issues related to computation offloading in edge computing. Based on the concepts of DQN and A3C, they proposed a deep reinforcement learning-based cloud-edge collaborative mobile computation offloading mechanism to jointly solve the decision-making and resource allocation problems. This work aims to minimize the weighted sum of the total delay and energy consumption of user terminals. The simulations were validated using a synthetic dataset and a network size consisting of up to 100 fog nodes and devices in industrial applications. According to the findings presented, the proposed algorithm showed a notable decrease in total cost from 25% to 60%, outperforming the other approaches. This outcome suggested that this scheme could significantly decrease energy consumption and achieve the objective in a short time frame. Correspondingly, the combination of low latency and energy efficiency signifies a well-balanced task offloading approach that can minimize the total cost and achieve the quickest convergence speed compared to other methods. Although this scheme outperforms the existing algorithm, it has some drawbacks. The first one, the scalability of the proposed method, is not

discussed for large-scale areas; they ignored the evaluation of the impact on the total cost when the number of user terminals increases. Moreover, simulation was applied to a few tasks, which could make the algorithm not guaranteed to handle computationally intensive tasks in complex scenarios. Finally, the authors neglected the heterogeneity of edge-cloud resources, limiting the application of their approach.

Similarly, the work in [87] used DQN method. The authors attempt to solve the computing offloading problem, which comprises two sub-problems: the task offloading decision and the resource allocation. To address this problem, they suggested DQN-based algorithms named Deep Reinforcement Learning-based Online Offloading (DROO) in MEC networks. The main objective of this paper is to minimize the computational time. In detail, this involves optimizing the transmission time allocation among wireless energy transfer(WPT), task offloading, and time allocation between different devices. In this case, to further optimize the delay for every task, the position of the task processing is decided, which can be executed locally or fully offloaded to an MEC server. The simulations were done in a network consisting of up to 100 fog nodes and up to 100 devices in mobile applications. According to the analysis presented, this algorithm decreased the execution latency from 0.81s to 0.059s. Hence, the latency decreased by 0.75s enabling real-time processing. This technique improves latency performance and enables the main objective to be reached in a short time-frame. Though the framework has strengths and can achieve optimal performance, it has its limits and may not be convenient from the implementation point of view. This paper is based on scenarios with a single agent. However, a single agent can pose scalability problems when faced with a large number of devices and fog nodes. Significantly, when the number of generated actions increases, the situation gets more complicated and the multi-agent method can better handle the problem of large action spaces than single-agent.

Another example is [90], in which the authors studied a task offloading and resource allocation problem in fog computing. To address this problem, they proposed a resource management approach based on contract theory and deep reinforcement learning technique (DQN) to find the optimal offloading strategy. The main objective of this method is to guarantee the QoS of the application, in terms of latency, task completion, and energy consumption. In this scenario, vehicles work as mobile fog nodes and assist in utilizing computing resources to improve user QoS satisfaction. To prevent decision collisions and interference in offloading tasks in multiple vehicles, they developed an approach based on the queueing theory. The simulation was conducted using a synthetic dataset on a network that with over 100 fog nodes and more than 100 devices in vehicular applications. After simulation work, the results showed that the suggested algorithm improved the QoS of vehicle applications and enhanced the performance of the system by around 20%. Although there are many advantages of the proposed method, on the other hand, it has some challenges, such as the authors did not take into account the security and mobility of vehicles, although it was a vehicular network. Then, they did not compare the proposed method with other Reinforcement Learning techniques. Therefore, the comparison should be done to demonstrate the benefits of this work.

In the same manner, in [91], the authors have paid attention to offloading decision based on DQN algorithm. To attempt this problem, they proposed a local differential-privacy-based deep reinforcement learning (LDP-DRL) that aims to enhance the quality of service (QoS) and satisfy the privacy requirements for task offloading. The QoS requirements are based on three parameters: the energy consumption cost, which includes the energy consumption of the offloading tasks to the ECSs and the energy consumption of the processing tasks; the time cost, including the transmission time and the completion time. A real dataset, generated through using real devices to test the DRL-based approaches, and the simulation were conducted on a network comprising over 1000 fog nodes and several devices in industrial applications. After extensive experiments to evaluate the performance of the proposed framework, the results are compared with greedy and DRL-non-LDP approaches. The simulation results showed that the proposed method outperforms the other algorithms, enhancing QoS regarding privacy and cost-effectiveness by around 20%. However, this approach is not compared with other RL techniques.

Another example is [93], in which the authors suggested a decision-making method based on DQN to jointly optimize task offloading and resource allocation. To address this problem, they proposed a deep reinforcement learning (DRL)-aided task offloading and resource allocation scheme known as TORA-DRL in a cloud-edge computing environment. The main objective of this method is to minimize the energy consumption of the entire system. Simulations were carried out on a network composed of over 100 fog nodes and several devices in mobile application. According to the findings presented, the proposed approach could significantly reduce energy consumption compared to the other existing schemes, with reduction of 50%. Hence, it improved the entire system's performance, converged fast, and minimized energy consumption. Although there are many advantages of the proposed method, on the other hand, it has some weaknesses; latency is not considered an essential problem in this algorithm, which may pose challenges for real-time scenarios. Moreover, despite the DRL scheme seems interesting, a

general concept is presented, and there is no validation of the approach in a real scenario, so it is necessary to validate it in a real situation.

6.1.2.4 DRQN As a variant of DQN, a DRQN method can be used to solve computation offloading problem. In References [85], the task offloading mechanism for fog networks is based on SDN architecture. In this work, the authors proposed an algorithm based on Deep recurrent Q-network (DRQN) for task offloading in general applications. They attempt to address the problem of computing offloading, which is divided into two sub-problems: the first one is task offloading decision, and the second is the resource allocation for multiple fog node framework. The main objective of this paper is to maximize processing tasks by satisfying their delay constraints. In this case, the task delay included the transmission latency, the waiting delay, and the processing delay. As per this strategy, the average task delay is reduced by choosing a neighboring fog node that minimizes the transmission latency and the waiting delay. The proposed approach was evaluated with simulations consisting of up to 100 fog nodes and multiple devices. Then, was tested based on a real data set collected using CPU processing densities and memory sizes that used real application data and a YouTube video data set. The analysis presented revealed that implementing the proposed algorithm leads to a significant reduction in average task delay, with a decrease of 2 s observed. This outcome indicates that task offloading is carried out within short deadlines and DRL-based task offloading approach can minimize the execution of computational tasks, resulting in faster task completion times than the other existing methods. However, this approach didn't consider the energy consumption. Moreover, the limitation of this work comes with the SDN architecture, considering that the task offloading mechanism is based on SDN architecture, which has several drawbacks, such as security, scalability, and reliability. While dealing with these various issues to solve the problems of task offloading in fog computing by using SDN-based solutions, this work remains in its beginning stages.

6.1.2.5 Dueling DDQN The authors of [88] mainly discuss the issues related computing offloading in mobile applications. This study suggests a new method called deep reinforcement learning (DRL)-based multi-user multitask hybrid computing offloading model (DRL-MHO) for offloading schemes. The main goal of this contribution is to minimize long-term system delay. For this reason, they implemented an hybrid offloading model that combined LSTM and dueling DDQN techniques. Finally, the proposed algorithm is benchmarked against four task offloading methods. The simulation results showed that the total system delay is reduced by around 40.8%, 31.1%, 24.2%, and 15.3%, respectively. As the analysis indicates, this model exhibits the lowest latency and the second-lowest device energy consumption of all the solutions considered. The model's ability to achieve the lowest latency suggests that it can find the appropriate offloading decision and execute the tasks quickly, thus completing the optimization goal in the lowest time frame. Moreover, the relatively second-lowest energy consumption suggests that the model saves device battery life. Overall, the offloading strategy implemented in this model is a well-balanced approach, likely contributing to achieving the objective promptly and significantly reducing long-term overall system delay. However, the proposed study focuses on handling the task offloading problem as a single objective optimization and concentrates only on minimizing latency while energy consumption is not addressed well enough. Furthermore, a general concept is presented, and the proposed approach needs to be validated in a real-world situation.

6.1.3 Hybrid

6.1.3.1 DDPG The DDPG method was first introduced in [80], where authors proposed a collaborative offloading framework in heterogeneous edge networks. This work addressed the computing offloading problem by employing the multi-agent DRL (MA-DRL) solution based on a deep deterministic policy gradient (DDPG) algorithm. Their research aims to improve the system's overall utility in terms of QoE perspective. Furthermore, they considered the latency and energy consumption critical parameters for determining the QoE. The proposed approach was evaluated using simulations with over 1000 fog nodes and 1000 devices and validated by simulations with real-life mobile wireless dataset obtained from Shanghai Telecom in mobile applications. The outcomes revealed that the average system utility of the proposed approach is 22.5%, 37.5%, and 43.6% higher than the other algorithms. Despite the numerical simulations showed that the cooperative framework could improve the system utility compared to the existing works based on the non-cooperative offloading schemes. This scheme is not scalable and has some challenges, as when the number of EDs increased and exceeded some thresholds, the system utility decreased. It also implies that the offloading latency increased and thus degraded the overall system utility.

Similarly, authors in [86], discussed the issues related to computation offloading using DDPG algorithm. They proposed an intelligent task offloading scheme with resource allocation based on DDPG algorithm to solve the problem and determine the best offloading strategy. The main goal of this study is to minimize the energy

consumption of all computing tasks. Therefore, they proposed an algorithm named Twin Delayed Deep Deterministic Policy Gradient-based Intelligent Computation Offloading (TD3PG-ICO) to address this problem. The evaluation involved simulations conducted in general applications on a network comprising up to 100 fog nodes and up to 100 devices then compared with other algorithms. After simulation work, the results showed that the suggested approach performed well in terms of convergence and enhanced the robustness of the overall system. It has been observed that the proposed method consumed less energy by an average of 15.53% and 6.41% than the other approaches. Despite this approach enhanced the performance of the system, they didn't consider the latency as an important factor.

The authors of [92] mainly studied the issues related to making decisions offloading. To address this problem, they proposed a distributed DRL-based computation offloading scheme in edge computing environments that used the DDPG technique. The main objective of this work is to improve the quality of experience (QoE). The DRL model chooses the offloading target based on a model that considers both the task service delay and load balancing. In this case, the algorithm also shares experiences between the agents and transmits priority experiences into the replay buffer to improve the system's overall performance. Simulations are carried out on an environment comprising up to 100 fog nodes and more than 100 devices in general applications. After simulation work, the outcome of the proposed algorithm showed a decrease of 0.1s in total service time and enhance load balancing compared with other DRL techniques. Correspondingly, it improved the quality of experience (QoE). The main negative point of this article is that this approach has not been validated in a real situation. Furthermore, the scalability of the proposed scheme is not discussed for large-scale areas, so it should be demonstrated by formal methods or through implementation in a real-world scenario.

6.1.3.2 SAC In [89], authors proposed a DRL-based task offloading scheme in fog computing to satisfy the QoS requirements. They used three DRL methods to address this issue, including DQN, DDPG, and SAC. The main objective of this paper is to improve the quality of service of users and to meet their needs by maximizing the average utility of resources by taking into account different parameters such as latency, energy consumption, task deadline, and priority, which measures the quality of service during offloading. To this end, they classified the tasks into critical, high-priority, and low-priority tasks. The proposed approach is evaluated with three different solutions based on deep reinforcement learning (DRL) and was tested using a dataset collected by the researchers from different Arduino devices. Simulations were done on an environment comprising up to 100 fog nodes and up to 100 devices. After experiments, the results showed that the proposed algorithm outperforms the other approaches. Specifically, it has been observed that 96.23% of tasks can satisfy the task deadline with an enhancement of 8.25%. Thus, the average resource utility is optimized, and the QoS requirements are satisfied. However, as a drawback, the authors did not consider the mobility in this framework and assumed that each network component is static. Moreover, they didn't validate the proposed scheme in a real-world environment.

7 Analytical Discussion

Existing studies using Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) techniques tackle offloading issues in fog computing have been critically analyzed in this survey. The articles analyzed were taken from diverse conferences and journals across multiple publishers, providing a good understanding of the current research landscape in this field.

- **RQ2(RL techniques):** Which type of RL and DRL techniques are utilized for computation offloading in the fog environment?

The analytical discussion is presented as follows. Figure 8, a pie chart, shows the percentages of RL and DRL algorithms used to address computation offloading problems in fog computing. It has been shown that the most popular technique is DQN, which is at the top of the ranking with a substantial 35% usage rate across the surveyed literature. The next most frequently used technique to address the computation offloading problem is DDPG; it comes in second position with 24% usage. A2C/A3C algorithms closely follow this result, occupying the third position with an 12% implementation frequency. Notably, the review highlights a few articles that propose using Q learning, Dueling DDQN, DRQN and SAC algorithms, each of 6% usage. Lastly, we find solutions based on SARSA this techniques is the least with a frequency of implementation of 5%. In this comparative analysis of the most commonly DRL techniques used to solve task offloading problems in the reviewed papers, we examine the two popular algorithms DQN and DDPG: To evaluate the effectiveness of DQN in minimizing energy and latency, we found that in the overall simulations based on the DQN technique, 3 out of 5 works performed well in reducing energy and latency simultaneously. We can, therefore, conclude that the DQN method was able to find solutions that addressed multiple objectives, particularly in complex and dynamic environments such as mobile

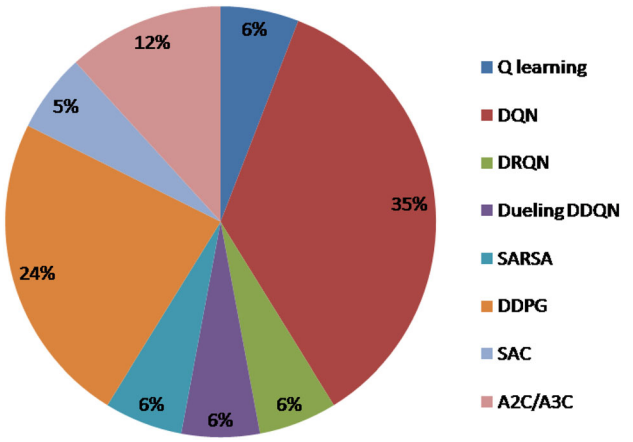

Fig. 8 Percentage of RL and DRL methods used in fog computing mechanism

applications, vehicular applications, and industrial applications. However, in simulations of work based on DDPG methods, the findings presented showed that the algorithm was capable of reducing significant latency. On the other hand, energy consumption was not always minimized, which suggests a certain limitation in the ability of DDPG to handle multiple objectives optimization. Hence, DQN may be a suitable Deep reinforcement learning method for solving computation offloading problems in dynamic and complex environments like the fog. It is adapted to for implementation in different use cases, including augmented reality, intelligent transportation and industrial applications, in order to enhance system performance, such as delay, energy, and cost.

- **RQ3(Objectives):** What performance objectives are optimized when solving computation offloading problems?

According to the objectives optimization, we identified four main performance metrics used for RL and DRL-based computation offloading, including minimizing latency, minimizing energy consumption, satisfying quality of service (QoS), and quality of experience QoE. We can conclude from this detailed study that two key performance metrics stand out above the rest: latency and energy consumption. In addition, minimizing latency and energy consumption emerge as the most critical metrics used to evaluate the performance of the system. When addressing challenges related to task offloading in the fog computing environment, where quick responsiveness and efficiency are paramount, it's perfectly reasonable to prioritize the lowest delay. Figure 9 presents the percentage of objectives used to address the computation offloading mechanisms based on RL and DRL techniques. As shown in the chart, minimizing latency of service is the dominant metric that appears in most of the articles reviewed, with 37% usage, while reducing the energy consumption of the overall system or user devices is addressed by many researchers, with a 31% usage rate across the surveyed literature. On the other hand, we find articles that consider quality of service QoS and quality of experience QoE as an optimization objective, which are the less frequently used measures. Specifically, the maximization of user QoS is only studied in a limited number of research papers, representing 19% of the total. Furthermore, the objectives related to QoE are the least frequent in the articles examined, accounting for 13% of the total. Overall, these findings suggest that less attention is paid to these metrics than latency and energy consumption.

- **RQ4(Use cases):** What use cases are used for RL and DRL-based offloading, and which environment specifications are considered in each use case, particularly in terms of device size, fog size and data capture?

In this part, we will discuss use cases considered in the RL and DRL-based offloading mechanisms in the fog system and environment specifications. In addition, we specify each use case in terms of data capture and network size, such as the size of fog nodes and devices. The existing case studies used in the implementation are general applications, mobile applications, vehicular applications, industrial applications, and smart villages applications. From the reviewed works, it can be concluded that mobile applications and general applications present the highest percentage by 35% and 35%, respectively, of the research papers, while industrial applications have been implemented with 14%. Finally, we found vehicular and smart village applications are the least explored. Regarding environment specification, we found that only 2 papers [80, 91] carried out simulations with more than 1000 fog nodes and 1000 fog devices in industrial and mobile applications. However,

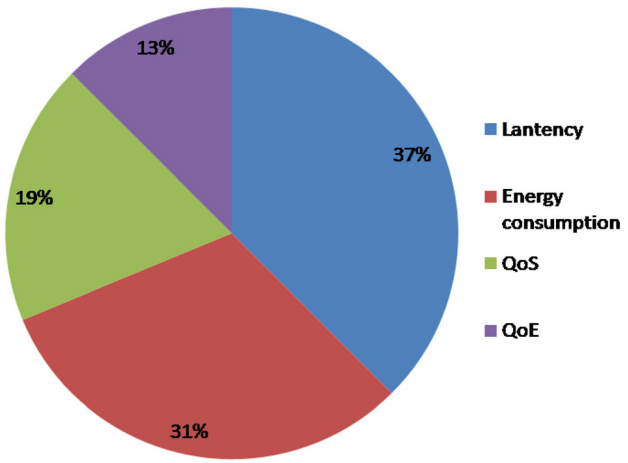

Fig. 9 Percentage of optimization objectives used in fog computing mechanism

the size of the networks simulated in the other works is not totally typical. In addition, simulations with a huge number of devices and fog nodes can be useful for testing the scalability of the system. On the other hand, it can be concluded that only 6 paper validated their approach using real datasets, and among these, few works carried out experiments on a real testbed. Implementing the proposed approach in the real world using testbeds is one of the most critical issues.

8 Open issues and Future research directions

In recent years, computation offloading in the fog computing environment, especially with Reinforcement Learning techniques, has received considerable interest from the research community. Despite the appreciable progress made in this area, many open issues still have to be tackled, which may lead future research to harvest the potential of RL to incorporate intelligence into the fog paradigm.

This section discusses some of the key challenges and investigates open issues concerning the implementation of LR-based approaches for an efficient task offloading to address computation offloading problems in fog computing environment. These challenges are summarised as follows:

- **Multi-objectives mechanisms:** Most previous works presented in Table 2 focus on handling a single objective, including latency, energy consumption, QoS, and QoE, without examining them in the same study. In addition, these contributions are simplified into a single objective optimization problem assigning a weight for each objective function to combine the multiple objective functions as a single objective function or taking objective functions to be optimized and the second objective as a constraint function. However, user satisfaction is one of the main criteria for measuring the performance of the computation offloading. In practice, considering just one performance objective is not feasible because the task offloading has various requirements. Therefore, user requirements must be satisfied. In the future, multiple objectives optimization should be studied when addressing the computing offloading problem. For example, jointly optimizing latency, energy consumption, and resource allocations using the DQN technique will be an exciting research topic.
- **Real-world evaluation:** Implementing the proposed offloading strategies in the real world guarantees that Quality of Service (QoS) requirements are satisfied. Nevertheless, carrying out real-world experiments in a fog computing environment poses challenges due to the time required for implementation and the high associated costs. Consequently, few works have validated their proposed approach using a real testbed. Most of the articles reviewed are evaluated based on simulation with synthetic datasets due to the challenges associated with real-world implementation. Hence, testing the optimization techniques with real-world experiments using real devices will enhance the applicability and importance of the research. On the other hand, most papers focus on addressing the task-offloading problem in general applications, such as UAV or IoT applications, which lack realism. Applying a real-world study in a specific field is still an open research area. Therefore, future work, can focus on addressing DQN-based task offloading problem in a real-world case in which we will tackle the field of smart agriculture, which aims to enhance performance in production and improve all aspects of traditional agriculture. Thus, the applications of IoT in agriculture will be exciting research field.
- **Security:** The most significant security challenge is developing strategies to enhance system robustness and strengthen protection against phishers. In the context of offloading in fog environments, large amounts of data generated by different end-user devices are outsourced to the fog resources for processing. Meanwhile, trust is absent between fog nodes and the end-user devices [94]. During the offloading process, end-user devices can send their data to neighboring fog nodes that have been hacked [95], which raises substantial cybersecurity challenges that impact the system performance. As a result, DRL-based computation offloading solutions should consider security and confidentiality to make the system robust and secure against attackers. DRL combined with Blockchain can address this problem [96, 97, 80]. Hence, it is recommended in future work that blockchain be integrated and combined with the proposed DRL approach to minimize the long-term system utility and maximize user privacy. This will solve the privacy issue and enhance the system performance.
- **Mobility:** In various research areas, including Vehicular Ad-hoc networks (VANETs) [98], Unmanned Aerial Vehicles (UAV) [99], Intelligent Transportation (ITS) [100], and mobile applications [101], devices exhibit dynamic behavior, moving rapidly between different places [102]. This dynamicity causes a new challenge for task offloading in edge computing, which can significantly affect the system performance. The main challenge is to have an effective mobility management approach to maintain the connection between the devices and the edge server, mainly when

leaving the initial location. As shown in Table 2, one of the drawbacks of previous studies is that they do not take vehicle mobility into account. Consequently, new approaches in future work can address optimization methodology based on DRL that consider the cost and device mobility [103].

9 Conclusion

The fog computing concept brings computation resources closer to the network edge. This development has paved the way for the end users to offload their computation-intensive and delay-sensitive applications to fog nodes, in order to meet the strict application requirements of latency and to reduce energy consumption at end users. However, designing an algorithm to decide which task should be offloaded is crucial. Due to the complexity of the systems and the dynamic environment, RL and DRL algorithms have gained momentum in recent years from the research community to solve offloading issues. In this paper, we presented a comprehensive and detailed survey related to the application of RL and DRL in computation offloading problems in fog environments. We proposed a taxonomy to classify various RL and DRL-based offloading mechanisms. Next, we answered five basic research questions. According to research questions 1 and 2, our proposed taxonomy classified the state-of-the-art studies based on RL techniques, organized according to three main categories: value-based, policy-based, and hybrid-based algorithms. Then, we compared these techniques to optimize the offloading decision; we considered various aspects such as utilized techniques, objectives to be achieved by the algorithm (minimizing latency, energy consumption, QoS, and QoE), architecture, and use cases. Then, in particular, we identify the advantages and weaknesses of each paper. This taxonomy will help researchers choose the optimal technique for an efficient task offloading, improving latency performance, and reducing power consumption. In response to research question 2, the analysis indicated that DQN was the most commonly used technique in the surveyed literature, with 35%. Research Question 3 highlighted that the most crucial metrics for evaluating offloading mechanisms were latency by 37% and energy consumption by 31% of usage. Regarding research question 4, the most common use cases utilized in the experiment results were mobile applications by 35% and general applications by 35%. Regarding environment specifications, recent research validates the results of the proposed approach in small size of networks, mostly in simplistic scenarios, while the tests under real assumptions have not been taken into account. Finally, considering the current gap in the literature, we highlighted some interesting research challenges associated with tasks offloading in fog computing and discussed future research directions that aim to accelerate the development in this area further.

Author contributions All authors reviewed the manuscript.

Funding The authors received no funding for this study.

Data availability The authors declare that there was no dataset exploited to conduct this survey.

Declarations

Competing interest The authors declare that they have no competing interests.

References

1. Shafique, K., Khawaja, B.A., Sabir, F., Qazi, S., Mustaqim, M.: Internet of things (iot) for next-generation smart systems: a review of current challenges, future trends and prospects for emerging 5g-iot scenarios. Ieee Access **8**, 23022–23040 (2020)
2. Armbrust, M., Fox, A., Griffith, R., Joseph, A.D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., et al.: A view of cloud computing. Commun. ACM **53**(4), 50–58 (2010)
3. Yi, S., Qin, Z., Li, Q.: Security and privacy issues of fog computing: A survey, pp. 685–695 (2015). https://doi.org/10.1007/978-3-319-21837-3_67
4. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the internet of things. In: Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, pp. 13–16 (2012)
5. Singh, J., Singh, P., Gill, S.S.: Fog computing: a taxonomy, systematic review, current trends and research challenges. J. Parallel Distrib. Comput. **157**, 56–85 (2021)
6. Mahmood, Z., Ramachandran, M.: Fog computing: concepts, principles and related paradigms. In: Mahmood, Z. (ed.) Fog Computing: Concepts, Frameworks and Technologies, pp. 3–21. Springer, New York (2018)
7. Jafari, V., Rezvani, M.H.: Joint optimization of energy consumption and time delay in iot-fog-cloud computing environments using nsga-ii metaheuristic algorithm. J. Ambient Intell. Hum. Comput. **14**(3), 1675–1698 (2023)
8. Mach, P., Becvar, Z.: Mobile edge computing: a survey on architecture and computation offloading. IEEE Commun. Surv. Tutor. **19**(3), 1628–1656 (2017)
9. Choudhury, A., Ghose, M., Islam, A., et al.: Machine learning-based computation offloading in multi-access edge computing: a survey. J. Syst. Architect. **148**, 103090 (2024)
10. Taheri-abed, S., Eftekhari Moghadam, A.M., Rezvani, M.H.: Machine learning-based computation offloading in edge and fog: a systematic review. Clust. Comput. **26**(5), 3113–3144 (2023)
11. Hortelano, D., Miguel, I., Barroso, R.J.D., Aguado, J.C., Merayo, N., Ruiz, L., Asensio, A., Masip-Bruin, X., Fernández, P., Lorenzo, R.M., et al.: A comprehensive survey on reinforcement-learning-based computation offloading techniques in edge computing systems. J. Netw. Comput. Appl. **216**, 103669 (2023)

12. Jiang, F., Dong, L., Wang, K., Yang, K., Pan, C.: Distributed resource scheduling for large-scale mec systems: a multiagent ensemble deep reinforcement learning with imitation acceleration. IEEE Internet Things J. **9**(9), 6597–6610 (2021)
13. Zhao, Z., Liang, Y., Jin, X.: Handling large-scale action space in deep q network. In: 2018 International Conference on Artificial Intelligence and Big Data (ICAIBD), pp. 93–96 (2018). IEEE
14. Hortelano, D., Miguel, I., Barroso, R.J.D., Aguado, J.C., Merayo, N., Ruiz, L., Asensio, A., Masip-Bruin, X., Fernández, P., Lorenzo, R.M., Abril, E.J.: A comprehensive survey on reinforcement-learning-based computation offloading techniques in edge computing systems. J. Netw. Comput. Appl. **216**(C) (2023)https://doi.org/10.1016/j.jnca.2023.103669
15. Abdulazeez, D.H., Askar, S.K.: Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment: A comprehensive review. IEEE Access (2023)
16. Saeik, F., Avgeris, M., Spatharakis, D., Santi, N., Dechouniotis, D., Violos, J., Leivadeas, A., Athanasopoulos, N., Mitton, N., Papavassiliou, S.: Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions. Comput. Netw. **195**, 108177 (2021). https://doi.org/10.1016/j.comnet.2021.108177
17. Tran-Dang, H., Bhardwaj, S., Rahim, T., Musaddiq, A., Kim, D.-S.: Reinforcement learning based resource management for fog computing environment: literature review, challenges, and open issues. J. Commun. Netw. **24**(1), 83–98 (2022)
18. Fahimullah, M., Ahvar, S., Trocan, M.: A review of resource management in fog computing: Machine learning perspective. arXiv preprint arXiv:2209.03066 (2022)
19. Iftikhar, S., Gill, S.S., Song, C., Xu, M., Aslanpour, M.S., Toosi, A.N., Du, J., Wu, H., Ghosh, S., Chowdhury, D., Golec, M., Kumar, M., Abdelmoniem, A.M., Cuadrado, F., Varghese, B., Rana, O.,: Artificial Intelligence, Cloud computing, Fog computing, Edge computing, Machine Learning, Internet of Things, Systematic Literature Review, S.D., Uhlig, S.: Ai-based fog and edge computing: A systematic review, taxonomy and future directions. Internet of Things 21, 100674 (2023) https://doi.org/10.1016/j.iot.2022.100674
20. Kumari, N., Yadav, A., Jana, P.K.: Task offloading in fog computing: a survey of algorithms and optimization techniques. Comput. Netw. **214**, 109137 (2022)
21. Shakarami, A., Ghobaei-Arani, M., Shahidinejad, A.: A survey on the computation offloading approaches in mobile edge computing: a machine learning-based perspective. Comput. Netw. **182**, 107496 (2020)
22. Nisha, P.: Fog computing and its real time applications. Int. J. Emerg. Technol. Adv. Eng. **5**(6), 266–269 (2015)
23. Binh, H.T.T., Anh, T.T., Son, D.B., Duc, P.A., Nguyen, B.M.: An evolutionary algorithm for solving task scheduling problem in cloud-fog computing environment. In: Proceedings of the Ninth International Symposium on Information and Communication Technology. SoICT 2018, pp. 397–404. Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3287921.3287984
24. Hardesty, L.: Fog computing group publishes reference architecture. https://www.sdxcentral.com/articles/news/Fog-computing-group-publishes-reference-architecture/2017/02/. Accessed: 2020-04-20 (2017)
25. AIT SALAHT, F., Desprez, F., Lebre, A.: An overview of service placement problem in Fog and Edge Computing. Research Report RR-9295, Univ Lyon, EnsL, UCBL, CNRS, Inria, LIP, LYON, France (October 2019). https://hal.inria.fr/hal-02313711
26. Afzali, M., Mohammad Vali Samani, A., Naji, H.R.: An efficient resource allocation of iot requests in hybrid fog–cloud environment. The Journal of Supercomputing 80(4), 4600–4624 (2024)
27. Laroui, M., Nour, B., Moungla, H., Cherif, M.A., Afifi, H., Guizani, M.: Edge and fog computing for iot: a survey on current research activities & future directions. Comput. Commun. **180**, 210–231 (2021)
28. Laghari, A.A., Jumani, A.K., Laghari, R.A.: Review and state of art of fog computing. Arch. Comput. Methods Eng. **28**(5), 1–13 (2021)
29. Mahmud, R., Kotagiri, R., Buyya, R.: Fog computing: A taxonomy, survey and future directions. Internet of Everything: Algorithms, Methodologies, Technologies and Perspectives, 103–130 (2018)
30. Sabireen, H., Neelanarayanan, V.: A review on fog computing: architecture, fog with iot, algorithms and research challenges. Ict Express **7**(2), 162–176 (2021)
31. Jamil, B., Shojafar, M., Ahmed, I., Ullah, A., Munir, K., Ijaz, H.: A job scheduling algorithm for delay and performance optimization in fog computing. Concurr. Comput. Pract. Exp. **32**(7), 5581 (2020). https://doi.org/10.1002/cpe.5581
32. Islam, A., Debnath, A., Ghose, M., Chakraborty, S.: A survey on task offloading in multi-access edge computing. J. Syst. Architect. **118**, 102225 (2021)
33. Attiya, H., Welch, J.: Distributed Computing: Fundamentals, Simulations, and Advanced Topics, vol. 19. Wiley, New York (2004)
34. Ding, H., Fang, Y.: Virtual infrastructure at traffic lights: vehicular temporary storage assisted data transportation at signalized intersections. IEEE Trans. Veh. Technol. **67**(12), 12452–12456 (2018). https://doi.org/10.1109/TVT.2018.2871414
35. Hu, P., Dhelim, S., Ning, H., Qiu, T.: Survey on fog computing: architecture, key technologies, applications and open issues. J. Netw. Comput. Appl. **98**, 27–42 (2017)
36. Ren, J., Zhang, D., He, S., Zhang, Y., Li, T.: A survey on end-edge-cloud orchestrated network computing paradigms: transparent computing, mobile edge computing, fog computing, and cloudlet. ACM Comput. Surv. (2019). https://doi.org/10.1145/3362031
37. Yi, S., Hao, Z., Qin, Z., Li, Q.: Fog computing: Platform and applications. In: 2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb), pp. 73–78 (2015). IEEE
38. Dastjerdi, A.V., Gupta, H., Calheiros, R.N., Ghosh, S.K., Buyya, R.: Fog computing: principles, architectures, and applications. In: Internet of Things, pp. 61–75. Elsevier, Amsterdam (2016)
39. Ghobaei-Arani, M., Souri, A., Rahmanian, A.A.: Resource management approaches in fog computing: a comprehensive review. J. Grid Comput. **18**(1), 1–42 (2020)
40. Labiod, Y., Amara Korba, A., Ghoualmi, N.: Fog computing-based intrusion detection architecture to protect iot networks. Wirel. Person. Commun. **125**(1), 231–259 (2022)
41. Ren, Y., Chen, C., Hu, M., Feng, G., Zhang, X.: Bfdac: A blockchain-based and fog computing-assisted data access control scheme in vehicular social networks. IEEE Internet of Things Journal (2023)
42. Yang, H., Guo, Y., Guo, Y.: Blockchain-based cloud-fog collaborative smart home authentication scheme. Comput. Netw. 110240 (2024)
43. Yi, S., Qin, Z., Li, Q.: Security and privacy issues of fog computing: A survey. In: Wireless Algorithms, Systems, and Applications: 10th International Conference, WASA 2015, Qufu, China, August 10-12, 2015, Proceedings 10, pp. 685–695 (2015). Springer

44. Yi, S., Li, C., Li, Q.: A survey of fog computing: concepts, applications and issues. In: Proceedings of the 2015 Workshop on Mobile Big Data, pp. 37–42 (2015)
45. Vahid Dastjerdi, A., Gupta, H., Calheiros, R.N., Ghosh, S.K., Buyya, R.: Fog computing: Principles, architectures, and applications. arXiv preprint arXiv:1601.02752 (2016)
46. Taneja, M., Davy, A.: Resource aware placement of iot application modules in fog-cloud computing paradigm. In: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), pp. 1222–1228 (2017). IEEE
47. Atlam, H.F., Walters, R.J., Wills, G.B.: Fog computing and the internet of things: a review. Big Data Cogn. Comput. **2**(2), 10 (2018)
48. Parveen, S., Singh, P., Arora, D.: Fog computing research opportunities and challenges: A comprehensive survey. In: Proceedings of First International Conference on Computing, Communications, and Cyber-Security (IC4S 2019), pp. 171–181 (2020). Springer
49. Li, W., Jin, S.: Performance evaluation and optimization of a task offloading strategy on the mobile edge computing with edge heterogeneity. J. Supercomput. **77**(11), 12486–12507 (2021)
50. Liu, Y., Mao, Y., Liu, Z., Ye, F., Yang, Y.: Joint task offloading and resource allocation in heterogeneous edge environments. IEEE Trans. Mob. Comput. (2023)
51. Mustafa, E., Shuja, J., Zaman, S.K., Jehangiri, A.I., Din, S., Rehman, F., Mustafa, S., Maqsood, T., Khan, A.N.: Joint wireless power transfer and task offloading in mobile edge computing: a survey. Clust. Comput. **25**(4), 2429–2448 (2022)
52. Kumari, N., Yadav, A., Jana, P.K.: Task offloading in fog computing: a survey of algorithms and optimization techniques. Comput. Netw. **214**, 109137 (2022)
53. Wang, B., Wang, C., Huang, W., Song, Y., Qin, X.: A survey and taxonomy on task offloading for edge-cloud computing. IEEE Access **8**, 186080–186101 (2020)
54. Saeik, F., Avgeris, M., Spatharakis, D., Santi, N., Dechouniotis, D., Violos, J., Leivadeas, A., Athanasopoulos, N., Mitton, N., Papavassiliou, S.: Task offloading in edge and cloud computing: a survey on mathematical, artificial intelligence and control theory solutions. Comput. Netw. **195**, 108177 (2021)
55. Gasmi, K., Dilek, S., Tosun, S., Ozdemir, S.: A survey on computation offloading and service placement in fog computing-based iot. J. Supercomput. **78**(2), 1983–2014 (2022)
56. Chen, M., Hao, Y.: Task offloading for mobile edge computing in software defined ultra-dense network. IEEE J. Sel. Areas Commun. **36**(3), 587–597 (2018)
57. Alameddine, H.A., Sharafeddine, S., Sebbah, S., Ayoubi, S., Assi, C.: Dynamic task offloading and scheduling for low-latency iot services in multi-access edge computing. IEEE J. Sel. Areas Commun. **37**(3), 668–682 (2019)
58. Yang, L., Zhang, H., Li, M., Guo, J., Ji, H.: Mobile edge computing empowered energy efficient task offloading in 5g. IEEE Trans. Veh. Technol. **67**(7), 6398–6409 (2018)
59. Gill, S.S., Xu, M., Ottaviani, C., Patros, P., Bahsoon, R., Shaghaghi, A., Golec, M., Stankovski, V., Wu, H., Abraham, A., et al.: Ai for next generation computing: emerging trends and future directions. Internet Things **19**, 100514 (2022)
60. Tuli, S., Gill, S.S., Garraghan, P., Buyya, R., Casale, G., Jennings, N.: Start: straggler prediction and mitigation for cloud computing environments using encoder lstm networks. IEEE Trans. Serv. Comput. **16**(1), 615–627 (2021)
61. Teoh, Y.K., Gill, S.S., Parlikad, A.K.: Iot and fog computing based predictive maintenance model for effective asset management in industry 4.0 using machine learning. IEEE Internet Things J. **10**(3), 2087–2094 (2021)
62. Bianchini, R., Fontoura, M., Cortez, E., Bonde, A., Muzio, A., Constantin, A.-M., Moscibroda, T., Magalhaes, G., Bablani, G., Russinovich, M.: Toward ml-centric cloud platforms. Commun. ACM **63**(2), 50–59 (2020)
63. Aljanabi, S., Chalechale, A.: Improving iot services using a hybrid fog-cloud offloading. IEEE Access **9**, 13775–13788 (2021)
64. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press, Cambridge (2018)
65. Sewak, M.: Deep Reinforcement Learning. Springer, New York (2019)
66. Puterman, M.L.: Markov Decision Processes: Discrete Stochastic Dynamic Programming. Wiley, New York (2014)
67. Tran-Dang, H., Bhardwaj, S., Rahim, T., Musaddiq, A., Kim, D.-S.: Reinforcement learning based resource management for fog computing environment: literature review, challenges, and open issues. J. Commun. Netw. **24**(1), 83–98 (2022)
68. Van Otterlo, M., Wiering, M.: Reinforcement learning and Markov decision processes. In: Reinforcement Learning, pp. 3–42. Springer, New York (2012)
69. Kanervisto, A., Scheller, C., Hautamäki, V.: Action space shaping in deep reinforcement learning. In: 2020 IEEE Conference on Games (CoG), pp. 479–486 (2020). IEEE
70. Kumar, A., Buckley, T., Lanier, J.B., Wang, Q., Kavelaars, A., Kuzovkin, I.: Offworld gym: open-access physical robotics environment for real-world reinforcement learning benchmark and research. arXiv preprint arXiv:1910.08639 (2019)
71. Chen, X., Liu, G.: Energy-efficient task offloading and resource allocation via deep reinforcement learning for augmented reality in mobile edge networks. IEEE Internet Things J. **8**(13), 10843–10856 (2021)
72. Cai, Q., Cui, C., Xiong, Y., Wang, W., Xie, Z., Zhang, M.: A survey on deep reinforcement learning for data processing and analytics. IEEE Trans. Knowl. Data Eng. **35**(5), 4446–4465 (2022)
73. Garnier, P., Viquerat, J., Rabault, J., Larcher, A., Kuhnle, A., Hachem, E.: A review on deep reinforcement learning for fluid mechanics. Comput. Fluids **225**, 104973 (2021)
74. Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Al Sallab, A.A., Yogamani, S., Pérez, P.: Deep reinforcement learning for autonomous driving: a survey. IEEE Trans. Intell. Transport. Syst. **23**(6), 4909–4926 (2021)
75. Xiong, Z., Zhang, Y., Niyato, D., Deng, R., Wang, P., Wang, L.-C.: Deep reinforcement learning for mobile 5g and beyond: fundamentals, applications, and challenges. IEEE Veh. Technol. Mag. **14**(2), 44–52 (2019)
76. Latif, S., Cuayáhuitl, H., Pervez, F., Shamshad, F., Ali, H.S., Cambria, E.: A survey on deep reinforcement learning for audio-based applications. Artif. Intell. Rev. **56**(3), 2193–2240 (2023)
77. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
78. Luong, N.C., Hoang, D.T., Gong, S., Niyato, D., Wang, P., Liang, Y.-C., Kim, D.I.: Applications of deep reinforcement learning in communications and networking: a survey. IEEE Commun. Surv. Tutor. **21**(4), 3133–3174 (2019)
79. Lei, L., Tan, Y., Zheng, K., Liu, S., Zhang, K., Shen, X.: Deep reinforcement learning for autonomous internet of things: model, applications and challenges. IEEE Commun. Surv. Tutor. **22**(3), 1722–1760 (2020)
80. Nguyen, D.C., Pathirana, P.N., Ding, M., Seneviratne, A.: Deep reinforcement learning for collaborative offloading in heterogeneous edge networks. In: 2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp. 297–303 (2021). IEEE
81. Bai, W., Qian, C.: Deep reinforcement learning for joint offloading and resource allocation in fog computing. In: 2021

IEEE 12th International Conference on Software Engineering and Service Science (ICSESS), pp. 131–134 (2021). IEEE
82. Chen, S., Chen, J., Miao, Y., Wang, Q., Zhao, C.: Deep reinforcement learning-based cloud-edge collaborative mobile computation offloading in industrial networks. IEEE Trans. Signal Inf. Process. Netw. **8**, 364–375 (2022)
83. Vemireddy, S., Rout, R.R.: Fuzzy reinforcement learning for energy efficient task offloading in vehicular fog computing. Comput. Netw. **199**, 108463 (2021)
84. Shahidinejad, A., Ghobaei-Arani, M.: Joint computation offloading and resource provisioning for e dge-cloud computing environment: a machine learning-based approach. Software **50**(12), 2212–2230 (2020)
85. Baek, J., Kaddoum, G.: Heterogeneous task offloading and resource allocations via deep recurrent reinforcement learning in partial observable multifog networks. IEEE Internet Things J. **8**(2), 1041–1056 (2020)
86. Chen, S., Tang, B., Wang, K.: Twin delayed deep deterministic policy gradient-based intelligent computation offloading for iot. Digit. Commun. Netw. **9**(4), 836–845 (2022)
87. Huang, L., Bi, S., Zhang, Y.-J.A.: Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. IEEE Trans. Mob. Comput. **19**(11), 2581–2593 (2019)
88. Cai, J., Fu, H., Liu, Y.: Deep reinforcement learning-based multitask hybrid computing offloading for multiaccess edge computing. Int. J. Intell. Syst. **37**(9), 6221–6243 (2022)
89. Jain, V., Kumar, B.: Qos-aware task offloading in fog environment using multi-agent deep reinforcement learning. J. Netw. Syst. Manag. **31**(1), 1–32 (2023)
90. Zhao, J., Kong, M., Li, Q., Sun, X.: Contract-based computing resource management via deep reinforcement learning in vehicular fog computing. IEEE Access **8**, 3319–3329 (2019)
91. Wei, D., Xi, N., Ma, X., Shojafar, M., Kumari, S., Ma, J.: Personalized privacy-aware task offloading for edge-cloud-assisted industrial internet of things in automated manufacturing. IEEE Trans. Ind. Inform. **18**(11), 7935–7945 (2022)
92. Park, J., Chung, K.: Distributed drl-based computation offloading scheme for improving qoe in edge computing environments. Sensors **23**(8), 4166 (2023)
93. Fang, C., Hu, Z., Meng, X., Tu, S., Wang, Z., Zeng, D., Ni, W., Guo, S., Han, Z.: Drl-driven joint task offloading and resource allocation for energy-efficient content delivery in cloud-edge cooperation networks. IEEE Trans. Veh. Technol. **72**(12), 16195–16207 (2023)
94. Abeshu, A., Chilamkurti, N.: Deep learning: the frontier for distributed attack detection in fog-to-things computing. IEEE Commun. Mag. **56**(2), 169–175 (2018)
95. Salami, Y., Khajehvand, V., Zeinali, E.: Sos-fci: a secure offloading scheme in fog-cloud-based iot. J. Supercomput. **80**(1), 570–600 (2024)
96. Zheng, X., Li, M., Chen, Y., Guo, J., Alam, M., Hu, W.: Blockchain-based secure computation offloading in vehicular networks. IEEE Trans. Intell. Transport. Syst. **22**(7), 4073–4087 (2020)
97. Nguyen, D.C., Pathirana, P.N., Ding, M., Seneviratne, A.: Privacy-preserved task offloading in mobile blockchain with deep reinforcement learning. IEEE Trans. Netw. Serv. Manag. **17**(4), 2536–2549 (2020)
98. Liang, L., Ye, H., Li, G.Y.: Toward intelligent vehicular networks: a machine learning framework. IEEE Internet Things J. **6**(1), 124–135 (2018)
99. Cheng, F., Zhang, S., Li, Z., Chen, Y., Zhao, N., Yu, F.R., Leung, V.C.: Uav trajectory optimization for data offloading at the edge of multiple cells. IEEE Trans. Veh. Technol. **67**(7), 6732–6736 (2018)
100. Li, Y., Yang, C., Chen, X., Liu, Y.: Mobility and dependency-aware task offloading for intelligent assisted driving in vehicular edge computing networks. Veh. Commun. **45**, 100720 (2024)
101. Lai, S., Huang, L., Ning, Q., Zhao, C.: Mobility-aware task offloading in mec with task migration and result caching. Ad Hoc Netw. **156**, 103411 (2024)
102. Zhou, J., Yang, Q., Zhao, L., Dai, H., Xiao, F.: Mobility-aware computation offloading in satellite edge computing networks. IEEE Trans. Mob. Comput. (2024). https://doi.org/10.1109/TMC.2024.3359759
103. Yang, C., Liu, Y., Chen, X., Zhong, W., Xie, S.: Efficient mobility-aware task offloading for vehicular edge computing networks. IEEE Access **7**, 26652–26664 (2019)

Takwa Allaoui is a PhD student in Telecommunications at the National Engineering School of Tunis (ENIT), Tunisia. She received the M.S. degree in Telecommunications from ENIT in 2021. Her research interests include computer networks, edge/fog/cloud computing, Artificial Intelligence (AI) and Internet of Things (IoT).

Kaouther Gasmi PhD., graduated from Tunis El Manar university. And research interests on parallel computing and optimization. She is an assistant professor of Dept. computer science at Carthage university.

Prof. Tahar Ezzedine received his M.S., Ph.D., and HDR (Accreditation to Supervise Research) degrees in Telecommunications from the prestigious National School of Engineers of Tunis (ENIT), Tunisia. Currently, he holds a Full Professorship in the Department of Information and Communication Technology at ENIT. He is also the founder and leader of an active R&D group at the SysCom Research Lab within ENIT. This group focuses on research and development in the areas of wireless sensor networks, the Internet of Things (IoT), and the application of Artificial Intelligence (AI) across various fields.